

SOFTWARE REQUIREMENTS SPECIFICATION

AI-Assisted E-Learning Platform

LMS Agentic AI — Requirements & System Design Document

DOCUMENT VERSION	1.0
PREPARED FOR	LMS Agentic AI Project
CLASSIFICATION	Internal / Technical
CODEBASE REFERENCE	lms_project (Django 6.1, PostgreSQL 16)

Table of Contents

1. [Introduction](#)
2. [Functional Requirements](#)
3. [Non-Functional Requirements](#)
4. [User Roles & RBAC Matrix](#)
5. [System Architecture](#)
6. [Database Design & ERD](#)
7. [User Journey Flows](#)
8. [AI & Agent Design](#)
9. [GitHub & Development Strategy](#)

1. Introduction

1.1 Project Title

AI-Assisted E-Learning Platform ("LMS Agentic AI") — a Django-based Learning Management System augmented with two role-scoped autonomous AI agents (a **Student Agent** and an **Instructor Agent**), a separate read-only **Chatbot**, and an adaptive-learning practice engine, backed by a normalized PostgreSQL schema and a resilient, multi-provider LLM transport layer.

1.2 Problem Statement

Traditional Learning Management Systems (Moodle-, Canvas-, and Blackboard-style platforms) share a set of structural limitations that this platform is explicitly designed to correct:

1. **Lack of personalized, adaptive learning.** Conventional LMS platforms serve every enrolled student the same static sequence of lessons, quizzes, and materials. There is no mechanism that continuously measures a student's per-topic competency and adjusts the difficulty, sequencing, or content of practice in response. A student who has mastered a topic is shown the same drills as a student who is struggling with it, and a student's weak areas are only visible to them retroactively, after a graded quiz, rather than being surfaced proactively.
2. **Delayed, manual quiz creation.** Authoring a quiz — writing multiple-choice distractors, marking correct answers, and (for open-ended items) designing essay prompts — is a slow, manual task that falls entirely on the instructor, even when the instructor has already uploaded a lecture document or slide deck that could ground that content programmatically. This creates a lag between when material is taught and when a formative assessment exists for it.
3. **Instructor workload for repetitive analysis.** Instructors must manually cross-reference gradebooks, quiz attempts, and per-student histories to identify at-risk students, class-wide weak topics, or which essay responses are outstanding. This does not scale past a handful of students per course and is rarely done proactively.
4. **Absence of proactive, reviewable study planning.** Students are rarely given a structured, prioritized plan of what to study next; where personalized recommendations exist in conventional systems, they are usually opaque ("recommended for you") without an auditable rationale and without any instructor-in-the-loop oversight of whether the plan is appropriate.
5. **Opaque AI behavior.** Where LMS vendors have begun bolting on "AI features," these are typically un-auditable black boxes: there is no record of what data an AI assistant read, what it changed, or under what permission it acted — a compliance and trust gap that any AI-augmented system operating on student academic records must close.

1.3 Project Objective

The platform's concrete, code-verifiable objectives are:

- **Integrate agentic AI as two separate, role-bound actors** (`agents.student_agent` , `agents.instructor_agent`) that read and mutate the domain model exclusively through a registry of individually permission-checked, individually audited tool functions (`agents/tools.py`), rather than through free-form database access from an LLM.
- **Personalize student practice** through an AI-Generated Adaptive Practice Mode (`planning.AdaptivePracticeSession`) that starts each student at a difficulty derived from their existing `TopicMastery` score and adjusts difficulty up after two consecutive correct answers or down after one incorrect answer, generating one new AI question at a time.
- **Streamline instructor management** by letting the Instructor Agent read an uploaded `Material` 's extracted text and generate a complete `Quiz` (mixed MCQ/essay, per `Question.question_type`) without manual authoring, generate a narrative course summary, and generate a remedial quiz targeted at the course's statistically hardest topic in a single tool call (`generate_remedial_quiz`).
- **Ensure real-time, closed-loop performance feedback** by recomputing `TopicMastery.mastery_percent` immediately after every completed `QuizAttempt` (`courses/services.py:update_mastery`) and surfacing it back to the student through `get_learning_dashboard` and to the instructor through `get_course_analytics` / `analyze_class_performance` .
- **Guarantee the platform degrades gracefully, never destructively**, when the underlying LLM provider is rate-limited, unreachable, or unconfigured, by routing every agent conversation through an offline, rule-based natural-language router (`agents/nlu.py`) that calls the exact same audited tool functions as the LLM path.

1.4 Target User Personas

Persona: Student (`accounts.User.role = "student"` , with `StudentProfile`)

- **Technical needs:** adaptive, difficulty-scaling practice sessions on a chosen topic; an AI-generated, instructor-reviewable study plan seeded automatically with prioritized `PracticeTask` rows; an on-demand AI tutor that can explain a topic, explain a specific lecture material, give an example, give a hint, or quiz them — grounded in the actual uploaded course material rather than generic web knowledge; immediate visibility into per-topic mastery and a personalized "Recommended for you" list; the ability to summarize an uploaded material in one of five selectable styles (`bullet_points` , `detailed` , `tldr` , `exam_prep` , `eli5`).
- **Data boundary:** every Student Agent tool operates only on rows where the calling user is the `student` foreign key, or on courses where the user appears in `courses_course_students` ; this is enforced twice — once implicitly by the tool set exposed to the role, and once explicitly inside every tool function.

Persona: Instructor (`accounts.User.role = "instructor"` , with `InstructorProfile`)

- **Technical needs:** AI quiz generation from an uploaded lecture document (PDF/DOCX/PPTX/TXT) or from a topic name directly (using all of a course's existing readable materials as grounding context, or a synthesized primer if none exist); automated grading assistance for essay/short-answer responses (`grade_essay_response` , taking a 0–1 score and free-text feedback); dashboards for class-wide analytics (score distribution, per-topic average mastery, recent attempts, outstanding essay grading count); a ranked at-risk-student list computed from average `TopicMastery` against a configurable threshold; the ability to review (approve/reject, with a note) a student-submitted `StudyPlan` before its practice tasks become the student's official plan.

- **Data boundary:** every Instructor Agent tool that touches a `Course`, `Quiz`, `Material`, or `QuestionResponse` filters on `course__instructor = user` (or the equivalent chain), so an instructor can never read or mutate another instructor's course through the agent layer.

Persona: Administrator (`accounts.User.role = "admin"` , `is_staff = True`)

- **Technical needs:** full visibility into the Django Admin (`django.contrib.admin`) over every model in every app; the ability to audit **every** agent tool invocation — successful, denied, errored, or awaiting confirmation — through `agents_agentactionlog`, including the exact input parameters and output/result JSON recorded for that call; enforcement that Role-Based Access Control is applied consistently regardless of whether a request originated from a dashboard button, the LLM tool-calling loop, or the offline NLU fallback router.
- **Data boundary:** admins are not given bespoke agent tools in this release; their oversight surface is the Django Admin and the `AgentActionLog` table, both read/write at the framework level per Django's standard staff/superuser permission model.

1.5 System Scope

In scope (implemented in the reviewed codebase):

- Django 6.1 monolith with six first-party apps: `accounts`, `courses`, `planning`, `agents`, `chatbot`, `notifications`, plus a `config` project package.
- Custom `AbstractUser`-based `User` model with a `role` field (`student` / `instructor` / `admin`) and 1-to-1 `StudentProfile` / `InstructorProfile` extension tables.
- Full course authoring hierarchy: `Course` → `Module` → `Lesson`, plus course-scoped `Topic` tags used to organize quizzes, mastery, study plans, and adaptive sessions.
- File-based `Material` uploads (PDF, DOCX, PPTX, TXT, video) with server-side text extraction (`pypdf`, `python-docx`, `python-pptx`) bounded to 40,000 characters per material, an `extraction_status` state machine (`done` / `unsupported` / `failed` / `n/a` for video), and per-material AI summarization cached on the `Material` row.
- A quiz engine supporting MCQ and essay question types, auto-grading for MCQ, manual instructor grading for essay/short-answer responses, and a `QuizAttempt.status` lifecycle (`in_progress` → `pending_review` or `graded`).
- A Student Agent and an Instructor Agent, each exposing a fixed, role-specific set of "tools" (Python functions with a JSON-Schema-described signature) that the LLM (or the offline router) may call; every call is written to `AgentActionLog` regardless of outcome.
- A destructive-action confirmation protocol: any tool marked `destructive=True` (`delete_practice_task`, `cancel_quiz`, `delete_material`) returns a confirmation prompt on its first call and only executes once called again with `confirm=true`.
- An AI-Generated Adaptive Practice Mode with server-persisted session state (`AdaptivePracticeSession`).
- A fully separate, read-only Chatbot (`chatbot` app) that has no tools, writes nothing to the database, and cannot invoke any `agents.tools` function — a deliberate architectural boundary between "answering" and "acting."

- A resilient LLM transport (`llm_client.py`) shared by all three AI surfaces, implementing a model fallback chain, a per-model rate-limit circuit breaker, and error classification (fatal auth errors vs. retryable server errors vs. retired-model errors).
- An **offline mode**: with no LLM key configured, an invalid key, or an exhausted quota, every agent tool remains reachable through a bilingual (English / Egyptian Arabic) rule-based intent router (`agents/nlu.py`) that resolves entities from the raw message text and calls the same tool functions, under the same RBAC and audit logging.
- Django's session-based authentication, CSRF protection, and the standard Django password validators.
- An in-app `Notification` system (unread/read) and a `AgentActionLog` -backed audit trail.
- A light/dark theme toggle persisted in browser `localStorage` and applied via a `data-theme` HTML attribute, with vanilla JavaScript (`static/js/dashboard.js` , `chat.js` , `theme.js`) — no HTMX or Alpine.js dependency is present in the codebase.
- PostgreSQL 16 as the primary datastore (via `docker-compose.yml`), with an explicit SQLite fallback switch (`DB_ENGINE=sqlite`) for environments without a running Postgres service.

Out of scope (not present in the reviewed codebase, and not to be implied by this SRS):

- Native mobile applications.
- Payment processing, subscription billing, or course marketplace features.
- Multi-tenant / multi-organization isolation (the schema assumes a single institution).
- Real-time video conferencing or live-lecture delivery (`Material.kind = "video"` supports only pre-recorded file upload and playback; it is explicitly excluded from AI text extraction and AI generation).
- SCORM/xAPI/LTI interoperability with external LMS platforms.
- A public REST/GraphQL API for third-party integration (the current surface is server-rendered Django views plus a small set of JSON endpoints consumed by the platform's own JavaScript).
- Fine-grained, per-object sharing (e.g., co-instructors on a single course); the schema models exactly one instructor per course (`courses_course.instructor_id` , N-1).
- Plagiarism detection, proctoring, or exam lockdown browser integration.

2. Functional Requirements

Every requirement below follows the IEEE syntax: "The system shall allow [User Role] to [Action] so that [Benefit]." Each requirement is traceable to a concrete model, view, or agent tool in the codebase.

2.1 User Authentication & Profile Management (`accounts`)

- **FR-2.1.1** The system shall allow an unauthenticated visitor to register a new account with a username, password, email, and role selection (student or instructor) so that they can gain access to role-appropriate features (`accounts/forms.py` , `accounts/views.py` , `templates/register.html`).
- **FR-2.1.2** The system shall allow a registered user to log in via Django's session-based authentication (`django.contrib.auth.views.LoginView`) so that their identity is established for every subsequent request without re-entering credentials.
- **FR-2.1.3** The system shall allow an authenticated user to log out, invalidating their session, so that their account cannot be used from a shared or abandoned browser session.
- **FR-2.1.4** The system shall allow the system to automatically create a `StudentProfile` or `InstructorProfile` row (1-to-1 with `User` , unique on `user_id`) upon registration according to the selected role, so that role-specific profile fields (`bio` , `level` for students; `bio` , `department` for instructors) are always available without a null-check at every read site.
- **FR-2.1.5** The system shall allow a student to view and edit their own `StudentProfile` (bio, level) so that their profile reflects their current standing.
- **FR-2.1.6** The system shall allow an instructor to view and edit their own `InstructorProfile` (bio, department) so that students and administrators can see their department affiliation.
- **FR-2.1.7** The system shall allow the system to expose a `role` property (`is_student` , `is_instructor`) on every authenticated `User` object so that every view, template conditional, and agent tool can perform a single, consistent role check (`accounts/models.py`).
- **FR-2.1.8** The system shall allow an administrator to manage any `User` , `StudentProfile` , or `InstructorProfile` record through the Django Admin so that account issues (lockouts, role corrections) can be resolved without direct database access.
- **FR-2.1.9** The system shall allow the system to redirect an authenticated user to their role-appropriate dashboard (`student_dashboard` or `instructor_dashboard`) immediately after login (`LOGIN_REDIRECT_URL = 'dashboard'`) so that the correct landing experience is shown without an extra click.

2.2 Course, Module, & Material Management (`courses`)

- **FR-2.2.1** The system shall allow an instructor to create a `Course` (title, description) so that they can begin organizing content for their students.

- **FR-2.2.2** The system shall allow the system to record exactly one instructor per course (`courses_course.instructor_id` , N-1 foreign key) so that ownership-based authorization checks (`course.instructor == user`) are unambiguous everywhere in the codebase.
- **FR-2.2.3** The system shall allow a student to enroll in a `Course` (adding a row to the `courses_course_students` join table) so that the course, its modules, lessons, quizzes, and materials become visible and actionable to them.
- **FR-2.2.4** The system shall allow an instructor to organize a course into `Module` records with an explicit `order` field so that content is presented in a deliberate sequence rather than database insertion order.
- **FR-2.2.5** The system shall allow an instructor to add `Lesson` records to a `Module` , each with `title` , `content` , and `order` , so that a module can be broken into individually addressable teaching units.
- **FR-2.2.6** The system shall allow an instructor to define `Topic` tags scoped to a course (unique per course by `name`) so that quizzes, questions, mastery scores, study-plan tasks, and adaptive-practice sessions can all be organized around a shared, granular curriculum vocabulary rather than the coarser `Module` / `Lesson` structure.
- **FR-2.2.7** The system shall allow an instructor to upload a `Material` (PDF, DOCX, PPTX, TXT, or video) optionally linked to a specific `Module` and/or `Lesson` , so that course content beyond the lesson text field is available to students (`agents.tools.upload_material`).
- **FR-2.2.8** The system shall allow the system to automatically classify an uploaded material's `kind` from its file extension and run text extraction against it (`courses/text_extraction.py`) so that the extracted text is available for AI summarization and quiz generation without a separate manual step.
- **FR-2.2.9** The system shall allow the system to record an `extraction_status` (`done` , `unsupported` , `failed` , or `n/a` for video) on every `Material` so that downstream AI tools can refuse gracefully with an explicit reason instead of silently operating on empty text.
- **FR-2.2.10** The system shall allow a student to mark a `Material` as done/not-done (`MaterialProgress` , unique per student/material pair) so that their material-completion percentage and study-streak calculations reflect what they have actually studied.
- **FR-2.2.11** The system shall allow an instructor to permanently delete a `Material` , including its underlying file on disk, so that outdated or incorrect content can be removed; this action shall be destructive and require explicit confirmation, and any `Quiz` previously generated from that material shall be retained as an independent record (`agents.tools.delete_material`).
- **FR-2.2.12** The system shall allow an instructor to trigger AI generation of a course-level narrative summary (student counts, module/lesson counts, weakest topics class-wide) and persist it on `Course.ai_summary` with a `Course.ai_summary_generated_at` timestamp, so that a human-readable course overview is always available without re-computation on every page load.

2.3 Assessment & Quiz Engine (`courses`)

- **FR-2.3.1** The system shall allow an instructor to manually create a `Quiz` scoped to a course and, optionally, a topic, so that they retain full manual authoring control alongside the AI-assisted paths.
- **FR-2.3.2** The system shall allow an instructor to manually add a `Question` (MCQ or essay) to any quiz they own, with MCQ questions requiring at least two `Choice` rows and exactly one marked `is_correct` , so that quiz integrity is enforced at the point of authoring rather than at grading time.

- **FR-2.3.3** The system shall allow the system to record whether a question was authored by a human or generated by AI (`Question.generated_by_ai`) so that provenance is preserved for audit and quality-review purposes.
- **FR-2.3.4** The system shall allow a student to start a `QuizAttempt` on any active quiz in a course they are enrolled in, transitioning it through the states `in_progress` → `pending_review` (if any essay answer is unscored) or `graded` (if every answer is auto-gradable), so that the grading workflow accurately reflects whether human input is still required.
- **FR-2.3.5** The system shall allow the system to automatically grade an MCQ `QuestionResponse` by comparing the student's `selected_choice` against the `Choice.is_correct` flag at submission time, so that students receive instant feedback on objective questions without instructor intervention.
- **FR-2.3.6** The system shall allow the system to flag an essay/short-answer `QuestionResponse` with `needs_review = true` so that it surfaces in the instructor's pending-grading queue until manually scored.
- **FR-2.3.7** The system shall allow an instructor to manually grade an essay response with a `manual_score` (0.0–1.0) and free-text `feedback`, recording `graded_by` and `graded_at`, so that subjective answers receive expert human judgment while remaining fully auditable (`agents.tools.grade_essay_response`).
- **FR-2.3.8** The system shall allow the system to recompute a `QuizAttempt.score` (0–100) whenever any of its `QuestionResponse` rows changes grading state, so that the attempt's aggregate score is always consistent with its constituent answers (`courses/services.py:_recompute_score`).
- **FR-2.3.9** The system shall allow the system to update the corresponding `TopicMastery.mastery_percent` and increment `attempts_count` immediately after a `QuizAttempt` is fully graded, so that a student's competency signal is always current for both the adaptive engine and instructor analytics (`courses/services.py:_update_mastery`).
- **FR-2.3.10** The system shall allow an instructor to cancel (deactivate) a quiz, setting `is_active = false` and recording `cancelled_at`, so that a flawed or obsolete quiz stops accepting new attempts without deleting its historical data; this action shall be destructive and require explicit confirmation (`agents.tools.cancel_quiz`).

2.4 Autonomous AI Student Agent & Practice Engine (`agents` , `planning`)

- **FR-2.4.1** The system shall allow a student to converse with the Student Agent in natural language (English or Egyptian Arabic) so that they can request any of the agent's sixteen supported actions without navigating a menu structure (`agents/student_agent.py`).
- **FR-2.4.2** The system shall allow the Student Agent to call `get_performance` so that the student can see their mastery percentage per topic across every enrolled course in one request.
- **FR-2.4.3** The system shall allow the Student Agent to call `explain_topic`, grounding the explanation in the course's uploaded materials that are actually relevant to that topic (or a synthesized primer if none exist), so that the tutoring response is accurate to the course content rather than generic.
- **FR-2.4.4** The system shall allow the Student Agent to call `ask_about_material` in one of six modes (`ask` , `explain_simply` , `give_example` , `explain_paragraph` , `hint` , `quiz_me`) scoped to exactly one uploaded lecture, so that a student reading a specific document receives answers grounded in that document alone.

- **FR-2.4.5** The system shall allow the Student Agent to call `summarize_material` in one of five selectable styles (`bullet_points` , `detailed` , `tldr` , `exam_prep` , `eli5`), producing a personalized, ephemeral summary that never overwrites the instructor's own cached `Material.ai_summary` , so that each student can consume content in the format that suits them.
- **FR-2.4.6** The system shall allow the Student Agent to call `generate_practice_quiz` , producing ungraded, answer-revealed self-check questions on a topic or practice task, so that a student can drill a weak area without the stakes (or database footprint) of a real graded quiz.
- **FR-2.4.7** The system shall allow the Student Agent to call `start_adaptive_practice` on a topic, which selects a starting difficulty (`easy` / `medium` / `hard`) from the student's existing `TopicMastery` score, generates one AI question, and persists an `AdaptivePracticeSession` , so that practice begins at an appropriate challenge level immediately.
- **FR-2.4.8** The system shall allow the Student Agent to call `submit_adaptive_answer` , grading the current question, incrementing difficulty after two consecutive correct answers, decrementing it after one incorrect answer, and generating the next question at the resulting difficulty, so that the session continuously calibrates to the student's demonstrated ability.
- **FR-2.4.9** The system shall allow the Student Agent to call `end_adaptive_practice` , deactivating the session and returning a summary (questions asked, questions correct, final difficulty), so that the student has closure on a practice session at any point.
- **FR-2.4.10** The system shall allow the Student Agent to call `create_study_plan` for an enrolled course, which computes the student's three weakest `TopicMastery` topics plus up to three never-attempted topics, assigns each a priority (`high` / `medium` / `low`), writes a structured multi-section summary, creates the `StudyPlan` in `pending` status, seeds one `PracticeTask` per focus topic, and notifies the course's instructor, so that the student receives an immediately actionable, evidence-based plan with zero manual setup.
- **FR-2.4.11** The system shall allow the Student Agent to call `create_practice_task` to add a task to an existing plan, `mark_task_done` to toggle a task's completion (fully reversible, no confirmation required), and `delete_practice_task` to remove a task (destructive, confirmation required), so that a student can maintain their own plan going forward.
- **FR-2.4.12** The system shall allow the Student Agent to call `recommend_quiz` , returning a random active quiz tagged with a given topic, so that the student has a concrete next assessment to take after studying.
- **FR-2.4.13** The system shall allow the Student Agent to call `mark_material_done` , toggling `MaterialProgress.is_done` and `done_at` , so that the material-completion count feeding the Learning Dashboard stays accurate.
- **FR-2.4.14** The system shall allow the Student Agent to call `get_learning_dashboard` , returning overall progress, quiz average, strongest/weakest topic, materials-completed count, and current study-streak in days, together with an AI-written narrative generated strictly from those computed numbers (so the prose can never contradict the statistics), so that a student gets both the data and a digestible interpretation of it.
- **FR-2.4.15** The system shall allow the Student Agent to call `get_recommendations` , returning a short, rule-based, ranked "Recommended for you" list where every entry carries a concrete, human-readable reason (a specific score, a wrong-answer count, or "never practiced"), so that recommendations are always explainable rather than an opaque AI suggestion.

- **FR-2.4.16** The system shall allow the system to reject any Student Agent tool call attempting to act on a course, topic, material, plan, or task the calling student does not own or is not enrolled in, returning a `PERMISSION_DENIED` result, so that a student can never read or mutate another student's data through the agent.

2.5 Autonomous AI Instructor Agent (`agents`)

- **FR-2.5.1** The system shall allow an instructor to converse with the Instructor Agent in natural language (English or Egyptian Arabic) so that they can perform any of the agent's seventeen supported actions conversationally (`agents/instructor_agent.py`).
- **FR-2.5.2** The system shall allow the Instructor Agent to call `get_course_analytics` , returning average score, a score-distribution histogram (four buckets), a per-topic average-mastery breakdown, the fifteen most recent graded attempts, and the count of essay responses still awaiting grading, for a course the instructor owns.
- **FR-2.5.3** The system shall allow the Instructor Agent to call `get_student_performance` for one named student within an owned course, returning their full attempt history and per-topic mastery, so that the instructor can prepare for a 1:1 conversation with concrete evidence.
- **FR-2.5.4** The system shall allow the Instructor Agent to call `analyze_class_performance` , which computes the class's statistically hardest topic, class average score, and at-risk-student count, then layers an AI-written recommended action on top of those exact numbers, so that the instructor gets both diagnosis and a next step in one call.
- **FR-2.5.5** The system shall allow the Instructor Agent to call `get_at_risk_students` , flagging every enrolled student whose average mastery across the course falls below a configurable threshold (default 50%) together with their single weakest topic, so that intervention can be targeted rather than blanket.
- **FR-2.5.6** The system shall allow the Instructor Agent to call `upload_material` , which creates the `Material` row and immediately runs text extraction, so that a file becomes AI-ready in the same conversational turn it is uploaded in.
- **FR-2.5.7** The system shall allow the Instructor Agent to call `generate_summary_from_material` , writing an AI summary of one uploaded material's extracted text back onto `Material.ai_summary` , so that every student sees a consistent, instructor-approved summary rather than each generating their own.
- **FR-2.5.8** The system shall allow the Instructor Agent to call `generate_quiz_from_material` , producing a mixed MCQ/essay quiz grounded in one specific material's extracted text and linking it back to that material via `Quiz.generated_from_material` , so that formative assessment can be produced the moment content is uploaded.
- **FR-2.5.9** The system shall allow the Instructor Agent to call `generate_quiz_for_topic` , producing an AI quiz for a named topic using every relevant, already-processed material in the course as combined context (or a synthesized topic primer if the course has no readable material yet), so that quiz generation is not blocked on picking one specific file.
- **FR-2.5.10** The system shall allow the Instructor Agent to call `generate_remedial_quiz` for a course, which — if no `topic_id` is supplied — automatically targets the course's current statistically hardest topic (as computed by `compute_class_performance`) and generates a quiz for it, so that remediation content can be produced in a single step immediately after diagnosing a class-wide weakness.

- **FR-2.5.11** The system shall allow the Instructor Agent to call `add_question_to_quiz` to manually author one additional MCQ or essay question onto an existing owned quiz, so that AI-generated quizzes can be manually extended without leaving the conversational interface.
- **FR-2.5.12** The system shall allow the Instructor Agent to call `grade_essay_response` with a 0–1 score and optional feedback, which updates the response, recomputes the parent attempt's score and status, and notifies the student, so that grading and student notification happen atomically from one call.
- **FR-2.5.13** The system shall allow the Instructor Agent to call `create_quiz` and `cancel_quiz` (the latter destructive, requiring confirmation) for full manual lifecycle control alongside the AI-generation tools.
- **FR-2.5.14** The system shall allow the Instructor Agent to call `delete_material` (destructive, requiring confirmation) to permanently remove an uploaded file and its `Material` record while preserving any quizzes already generated from it.
- **FR-2.5.15** The system shall allow the Instructor Agent to call `send_announcement`, broadcasting a message-prefixed `Notification` to every student enrolled in an owned course in one call, so that class-wide communication does not require iterating students by hand.
- **FR-2.5.16** The system shall allow the Instructor Agent to call `generate_course_summary`, writing an AI-narrated overview of the course's size and weakest topics back onto `Course.ai_summary`, so that a current, human-readable course snapshot is always available.
- **FR-2.5.17** The system shall allow the system to reject any Instructor Agent tool call attempting to act on a course, quiz, material, response, or student not associated with the calling instructor's own courses, returning a `PERMISSION_DENIED` result, so that one instructor can never read or mutate another instructor's course through the agent.

2.6 Study Plan & Adaptive Session Generator (`planning`)

- **FR-2.6.1** The system shall allow an instructor to call `review_study_plan` to approve or reject a `pending StudyPlan` submitted by one of their enrolled students, recording `reviewed_by`, `reviewed_at`, and an optional `review_note`, so that AI-generated study plans remain under human instructional oversight rather than being auto-approved.
- **FR-2.6.2** The system shall allow the system to reject a `review_study_plan` call if the target student is not enrolled in any course the calling instructor teaches, so that plan review respects course ownership boundaries even though `StudyPlan` itself is not directly course-scoped.
- **FR-2.6.3** The system shall allow the system to notify a student in-app the moment their plan's status changes (approved or rejected), including the reviewer's note if one was given, so that the review outcome is delivered without the student needing to poll for it.
- **FR-2.6.4** The system shall allow the system to persist an `AdaptivePracticeSession`'s full mutable state — current difficulty, streak counters, the current question text and choices as JSON, and running question/correct counts — as a database row rather than transient session/cache state, so that the adaptive engine's state survives across requests, browser tabs, and process restarts, and is queryable for audit.
- **FR-2.6.5** The system shall allow the system to guarantee at most one `AdaptivePracticeSession` per student is `is_active = true` at any time by deactivating any prior active session at the start of `start_adaptive_practice`, so that a student is never answering two concurrent question streams.

- **FR-2.6.6** The system shall allow a `PracticeTask` to optionally reference a `recommended_quiz` (nullable FK to `Quiz`), so that a plan's checklist item can point directly to the assessment that validates that topic once studied.
- **FR-2.6.7** The system shall allow the system to track `PracticeTask.is_done` independently of the parent `StudyPlan.status`, so that a student can make and see progress on their checklist while the plan itself is still `pending` instructor review.

2.7 Real-time Notifications & Action Logging (`notifications` , `agents`)

- **FR-2.7.1** The system shall allow the system to create a `Notification` (message, `is_read`, `created_at`) for a user whenever a triggering domain event occurs — a study plan is submitted (notifies the instructor), a study plan is reviewed (notifies the student), an essay response is graded (notifies the student), or an announcement is sent (notifies every enrolled student) — so that both roles stay informed without manually refreshing dashboards (`notifications/services.py`).
- **FR-2.7.2** The system shall allow an authenticated user to view their notifications and mark them as read, so that their unread-notification indicator accurately reflects what they have and have not seen.
- **FR-2.7.3** The system shall allow the system to make the current unread-notification count available to every rendered template via a Django context processor (`notifications.context_processors.notifications`), so that the notification indicator is consistent across every page without per-view boilerplate.
- **FR-2.7.4** The system shall allow the system to write exactly one `AgentActionLog` row for every agent tool invocation, regardless of its outcome (`success`, `denied`, `error`, or `pending_confirm`), recording the calling user, the agent (`student_agent` / `instructor_agent`), the tool name, a JSON-safe copy of the input parameters, a JSON-safe copy of the result, and a timestamp, so that a complete, tamper-evident audit trail of every AI-initiated action exists independent of which transport (LLM tool-call loop or offline NLU router) served the request.
- **FR-2.7.5** The system shall allow the system to write an `AgentActionLog` row with status `pending_confirm` — including the human-readable confirmation prompt as the result — the first time a destructive tool is called without `confirm=true`, so that even an unconfirmed, no-op attempt at a destructive action is captured in the audit trail.
- **FR-2.7.6** The system shall allow an administrator to query `AgentActionLog` (via the Django Admin or direct database access) filtered by user, agent, tool name, status, or timestamp range, so that any reported incident involving AI-driven data changes can be fully reconstructed.

3. Non-Functional Requirements

3.1 Performance

- **NFR-3.1.1** LLM tool-calling turns shall be bounded by `run_tool_loop`'s `max_iters=4` ceiling, so that a single conversational request can never enter an unbounded reasoning/tool loop against the model provider.
- **NFR-3.1.2** Any single inline retry sleep against the LLM provider shall not exceed `MAX_INLINE_SLEEP_SECONDS = 4.0` seconds (`llm_client.py`); a retry delay larger than this (interpreted as a daily-quota exhaustion rather than a per-minute limit) shall instead trigger a cooldown of that model and an immediate move to the next model in the chain or to the offline router, so that a rate-limited request never stalls a user-facing page for more than a few seconds.
- **NFR-3.1.3** A model placed on cooldown by the circuit breaker (following a 429 response) shall be skipped entirely — with zero network calls — for the duration of that cooldown (the provider-specified retry delay, or 60 seconds by default), so that a known-exhausted model's quota is never wasted on doomed requests.
- **NFR-3.1.4** When every model in the configured chain is on cooldown or has been permanently retired, the system shall fall back to the offline rule-based router in well under 100 ms (no network I/O is attempted), so that degraded mode never degrades response latency.
- **NFR-3.1.5** Text extracted from any uploaded `Material` shall be truncated to a maximum of 40,000 characters (`text_extraction.MAX_CHARS`) before being persisted or used as LLM context, so that prompt size, database row size, and downstream generation latency remain bounded regardless of source document length.
- **NFR-3.1.6** Database access patterns for list/aggregate agent tools (e.g., `get_course_analytics`, `analyze_class_performance`, `get_at_risk_students`) shall use `select_related / values().annotate()` aggregation at the ORM level rather than per-row Python loops issuing individual queries, so that analytics endpoints scale sub-linearly with enrolled-student and attempt counts.
- **NFR-3.1.7** File uploads for `Material` shall be supported up to Django's configured in-memory/body limits of 300 MB (`FILE_UPLOAD_MAX_MEMORY_SIZE`, `DATA_UPLOAD_MAX_MEMORY_SIZE` in `config/settings.py`), sized specifically to accommodate lecture video files.

3.2 Security

- **NFR-3.2.1** Authentication shall use Django's built-in session-based authentication (`django.contrib.sessions`, `django.contrib.auth`); the `AUTH_USER_MODEL` is the project's custom `accounts.User`, and `LOGIN_URL` / `LOGIN_REDIRECT_URL` / `LOGOUT_REDIRECT_URL` are explicitly configured so that every unauthenticated request to a protected view is redirected predictably.
- **NFR-3.2.2** Cross-Site Request Forgery protection shall be enforced globally via `django.middleware.csrf.CsrfViewMiddleware`, present in `MIDDLEWARE` ahead of the authentication middleware, so that every state-changing POST request (including agent chat and action endpoints) requires a valid CSRF token.
- **NFR-3.2.3** Every agent tool function shall perform an explicit, in-function role check (`if not user.is_student: raise ToolPermissionError(...)`, respectively `is_instructor`) as defense in depth on top of the coarser-grained tool set already exposed only to the matching role's agent, so that RBAC does not rely

solely on which conversational endpoint the request arrived through.

- **NFR-3.2.4** Every agent tool that operates on a specific `Course`, `Quiz`, `Material`, `Topic`, `StudyPlan`, `PracticeTask`, or `QuestionResponse` shall filter its database lookup by the calling user's ownership or enrollment (e.g., `Course.objects.filter(id=..., instructor=user)`, `Course.objects.filter(id=..., students=user)`) rather than fetching by primary key alone, so that Insecure Direct Object Reference (IDOR) is structurally prevented at the ORM query level, not by a secondary permission check.
- **NFR-3.2.5** LLM prompt-injection defense is achieved architecturally: the LLM is never granted raw database or filesystem access. Every action the model can take is mediated through a fixed, explicitly declared `TOOL_SPECS` JSON-Schema registry (`agents/student_agent.py`, `agents/instructor_agent.py`), and every tool re-validates its own arguments and the caller's authorization independently of anything the model asserts in its reasoning or tool-call arguments — so that adversarial content embedded in an uploaded `Material`'s extracted text (used as LLM context for summarization/quiz generation) cannot cause the model to take an unauthorized action; at worst it can only influence the wording of a generated summary or quiz question, which remains subject to the same downstream validation as any AI-generated content.
- **NFR-3.2.6** Tool sandboxing is enforced via the `@agent_tool` decorator (`agents/decorators.py`), which wraps every tool function so that (a) `ToolPermissionError` and `ToolValidationError` are caught and converted into a uniform `{"ok": False, "error": ..., "code": ...}` shape rather than an unhandled exception surfacing to the model or the HTTP response, and (b) any unexpected exception is caught, logged, and returned as a generic `{"ok": False, "error": "Internal tool error.", "code": "ERROR"}`, so that internal stack traces, query details, or file paths are never leaked back through the agent conversation.
- **NFR-3.2.7** Destructive tools (`delete_practice_task`, `cancel_quiz`, `delete_material`) shall require an explicit `confirm=true` parameter on a second, repeated call before executing, with the first (unconfirmed) call short-circuiting to a confirmation prompt and logging a `pending_confirm` audit row, so that an LLM's own initiative — or a prompt-injection attempt embedded in course content — cannot cause an irreversible action to execute on a single inferred call.
- **NFR-3.2.8** Input sanitization for uploaded files shall include extension-based `kind` classification (`text_extraction.guess_kind`) and per-format extraction routines that catch all exceptions and return an explicit `failed` status rather than propagating a parser exception, so that a malformed or malicious PDF/DOCX/PPTX cannot crash the request or leave a `Material` row in an inconsistent state.
- **NFR-3.2.9** LLM API credentials (`GEMINI_API_KEY`) shall be read exclusively from environment variables (via `python-dotenv` and `os.environ`), never hard-coded or logged; a `401 / 403` response from the provider shall be classified as a fatal authentication error and shall halt the model chain immediately without retrying, so that a revoked or invalid key fails fast and visibly (surfaced via `python manage.py ai_status --test`) rather than silently degrading.
- **NFR-3.2.10** The Django `SECRET_KEY` and `DEBUG` flag shall be environment-configurable (`config/settings.py`), so that production deployments can supply a unique secret key and disable debug mode without a code change.

3.3 Usability

- **NFR-3.3.1** The application layout shall be responsive, rendered server-side with Django templates (`templates/base.html` and per-view templates) and a shared stylesheet (`static/css/main.css`), so that dashboards, chat panels, and analytics pages remain usable across common desktop and tablet viewport widths.
- **NFR-3.3.2** A light/dark theme preference shall be persisted client-side in `localStorage` under the key `lms-theme` and applied via a `data-theme` attribute set on `<html>` before first paint (an inline `<script>` in `templates/base.html` runs prior to stylesheet application), so that returning users see their chosen theme immediately with no flash of the wrong theme.
- **NFR-3.3.3** Dynamic, in-page interactivity (the AI chat widget, dashboard data refreshes, theme toggling) shall be implemented with plain JavaScript (`static/js/chat.js` , `dashboard.js` , `theme.js`) issuing `fetch()` calls against JSON endpoints, with no HTMX or Alpine.js dependency present in the codebase, so that the client-side footprint stays minimal and framework-free.
- **NFR-3.3.4** The Student and Instructor Agent chat interfaces shall reply in the same language the user wrote in, including Egyptian Arabic, both on the LLM path (via an explicit system-prompt instruction) and on the offline NLU fallback path (`agents/nlu.py` normalizes Arabic diacritics, tatweel, and interchangeable letter forms), so that non-English-speaking users are not forced to switch languages to use the AI features.
- **NFR-3.3.5** When a destructive action requires confirmation, the system shall surface a clear, human-readable prompt (e.g., *"delete_material' is a destructive action. Call again with confirm=true to proceed."*) through the conversational interface, so that the user understands exactly what will happen before it does.
- **NFR-3.3.6** A degraded-mode warning shall be shown to the user only when the offline router could not actually satisfy the request (i.e., no tool call resulted); when the offline router successfully routes and executes the correct tool, no apology or warning banner shall be shown, so that "AI is degraded" messaging never obscures a request that in fact succeeded.

3.4 Reliability

- **NFR-3.4.1** Every LLM call across all three AI surfaces (Student Agent, Instructor Agent, Chatbot, and `courses.ai_generation`) shall route through the single shared transport in `llm_client.py` , so that retry policy, model fallback, and error classification are implemented and tested exactly once rather than duplicated (and potentially inconsistently duplicated) across call sites.
- **NFR-3.4.2** On any LLM provider failure — missing key, invalid key, network failure, rate limiting, or every model in the chain being on cooldown — `LLMUnavailableError` shall be raised and caught by the calling agent, which shall transparently fall back to the offline rule-based router (`agents/nlu.py`) rather than returning an HTTP 500 or a broken page to the user, so that an LLM provider outage degrades the AI experience without breaking the application.
- **NFR-3.4.3** A `404` (model retired) response from the provider shall cause that model name to be permanently removed from the in-process model chain for the remainder of that process's lifetime, so that a renamed/retired upstream model does not repeatedly incur a failed-request penalty on every subsequent call.
- **NFR-3.4.4** A `5xx` response from the provider shall be treated as transiently retryable (short backoff, bounded by NFR-3.1.2) before moving to the next model in the chain, distinct from the non-retried handling of `401` / `403` / `404` , so that genuinely transient server errors are not conflated with permanent failures.

- **NFR-3.4.5** An operator shall be able to force fully offline behavior via the `AI_OFFLINE_MODE=1` environment variable, bypassing the LLM entirely and running exclusively on the rule-based routers, so that a live demonstration, automated test run, or network-restricted environment can guarantee deterministic behavior independent of any external API's availability.
- **NFR-3.4.6** An operator shall be able to inspect current AI subsystem health via the `python manage.py ai_status` management command (with an optional `--test` flag that issues a real request to verify the configured key), so that AI availability issues can be diagnosed without reading application logs.
- **NFR-3.4.7** Every quiz-grading and mastery-recomputation operation that spans multiple related rows (`QuestionResponse` → `QuizAttempt.score / status` → `TopicMastery`) shall be performed via the service-layer functions in `courses/services.py` so that the operation is atomic and consistent (all dependent aggregates recomputed together) rather than left to callers to orchestrate manually, reducing the risk of a partially updated grading state if an operation is interrupted.
- **NFR-3.4.8** Every agent tool's failure path shall log a warning (via Python's `logging` module, configured in `config/settings.py` to always surface `agents`, `courses`, and `chatbot` logger output to the console at `INFO` level or above) rather than failing silently, so that degraded-mode transitions are observable by operators even when they are invisible to end users.

3.5 Data Consistency

- **NFR-3.5.1** The database shall be PostgreSQL 16 by default (`config/settings.py`, `docker-compose.yml`), a fully ACID-compliant RDBMS, so that every multi-row write performed by the service layer and agent tools (grading, mastery recomputation, plan/task seeding) is transactionally consistent under Django's default `atomic` request-level behavior and explicit `transaction.atomic()` blocks where used.
- **NFR-3.5.2** Every foreign key in the schema shall declare an explicit `on_delete` policy appropriate to its semantics: `CASCADE` for strictly dependent child rows (e.g., `Module.course`, `Lesson.module`, `Choice.question`, `PracticeTask.plan`, `AgentActionLog.user`), and `SET_NULL` for optional cross-references whose parent's deletion should not cascade destructively (e.g., `Quiz.generated_from_material`, `PracticeTask.recommended_quiz`, `StudyPlan.reviewed_by`, `QuestionResponse.graded_by`), so that referential integrity is enforced by the database itself, not application-level discipline alone.
- **NFR-3.5.3** Composite uniqueness constraints shall be enforced at the database level for `MaterialProgress` (unique on `student` + `material`) and `TopicMastery` (unique on `student` + `topic`), so that a student can have at most one progress/mastery row per material or topic regardless of concurrent requests or application-level race conditions.
- **NFR-3.5.4** `Topic.name` shall be enforced unique within its parent `course`, so that a course's topic taxonomy cannot silently fragment into duplicate near-identical entries that would split mastery, quiz, and study-plan data across them.
- **NFR-3.5.5** The schema shall satisfy Third Normal Form (3NF): every non-key attribute is fully functionally dependent on its table's primary key and not transitively dependent through another non-key attribute (see §6.4 for the full normalization rationale, including the `courses_course_students` join table used to resolve the Course ↔ Student many-to-many relationship without repeating group data).

- **NFR-3.5.6** JSON-typed columns (`AgentActionLog.params` , `AgentActionLog.result` , `AdaptivePracticeSession.current_choices`) shall only ever be written through the `_safe()` sanitization helper (`agents/decorators.py`) or an equivalent explicit serialization, converting any non-JSON-primitive value to its string representation, so that a value that cannot be represented in the column's JSON type can never cause a write failure that would otherwise abort an audit-log write mid-transaction.

4. User Roles & RBAC Matrix

4.1 Role Definitions

Role	Backing Field	Description
Student	<code>accounts_user.role = "student", is_student == True</code>	Enrolls in courses, consumes materials, takes quizzes, and is the exclusive caller of the Student Agent's tool set.
Instructor	<code>accounts_user.role = "instructor", is_instructor == True</code>	Owns courses (<code>courses_course.instructor_id</code>), authors/curates content and assessments, and is the exclusive caller of the Instructor Agent's tool set.
Administrator	<code>accounts_user.role = "admin", is_staff = True</code>	Full Django Admin access across all models; sole reader of the complete, unfiltered <code>AgentActionLog</code> audit trail. Does not call agent tools in this release.
AI Agent (Student Agent / Instructor Agent)	N/A (a code actor, not a <code>User</code> row)	Acts strictly <i>as</i> the authenticated Student or Instructor that invoked it — it has no identity or permissions of its own; every tool call is authorized against the underlying human user's role and ownership, and logged against that user's <code>AgentActionLog</code> .

4.2 Permission Matrix

Legend: **C** = Create, **R** = Read, **U** = Update, **D** = Delete, **X** = Execute Tool (i.e., invoke an AI agent action against this module), **—** = No access.

Module / Resource	Student	Instructor	Admin
Own <code>User</code> account & profile	R, U	R, U	C, R, U, D
Other users' accounts	—	—	C, R, U, D
<code>Course</code> (own/enrolled)	R (enrolled only), X	C, R, U, D (owned only), X	C, R, U, D (all)
<code>Course</code> (not own/enrolled)	—	—	C, R, U, D
<code>Module</code> / <code>Lesson</code>	R (via enrolled course)	C, R, U, D (owned course only)	C, R, U, D
<code>Topic</code>	R (via enrolled course)	C, R, U, D (owned course only), X	C, R, U, D
<code>Material</code> (upload/manage)	R, U (progress only), X	C, R, U, D, X (owned course only)	C, R, U, D
<code>Quiz</code> / <code>Question</code> / <code>Choice</code>	R, X (attempt only)	C, R, U, D, X (owned course only)	C, R, U, D
<code>QuizAttempt</code> / <code>QuestionResponse</code>	C, R (own only)	R, U (grading, owned course only), X	R, U, D
<code>TopicMastery</code>	R (own only)	R (own students only)	R
<code>StudyPlan</code>	C, R, U (own only), X	R, U (review: approve/reject, own students only), X	R, U, D

Module / Resource	Student	Instructor	Admin
PracticeTask	C, R, U, D (own plans only), X	R (own students' plans)	R, U, D
AdaptivePracticeSession	C, R, U (own only), X	—	R
AgentActionLog	R (own actions only, via own agent UI)	R (own actions only, via own agent UI)	R (all, unfiltered)
Notification	R, U (own only)	R, U (own only), X (send: owned course students)	R, U, D
Chatbot (read-only Q&A)	X (own data only)	X (own data only)	—
Django Admin site	—	—	Full access

4.3 Notes on RBAC Enforcement

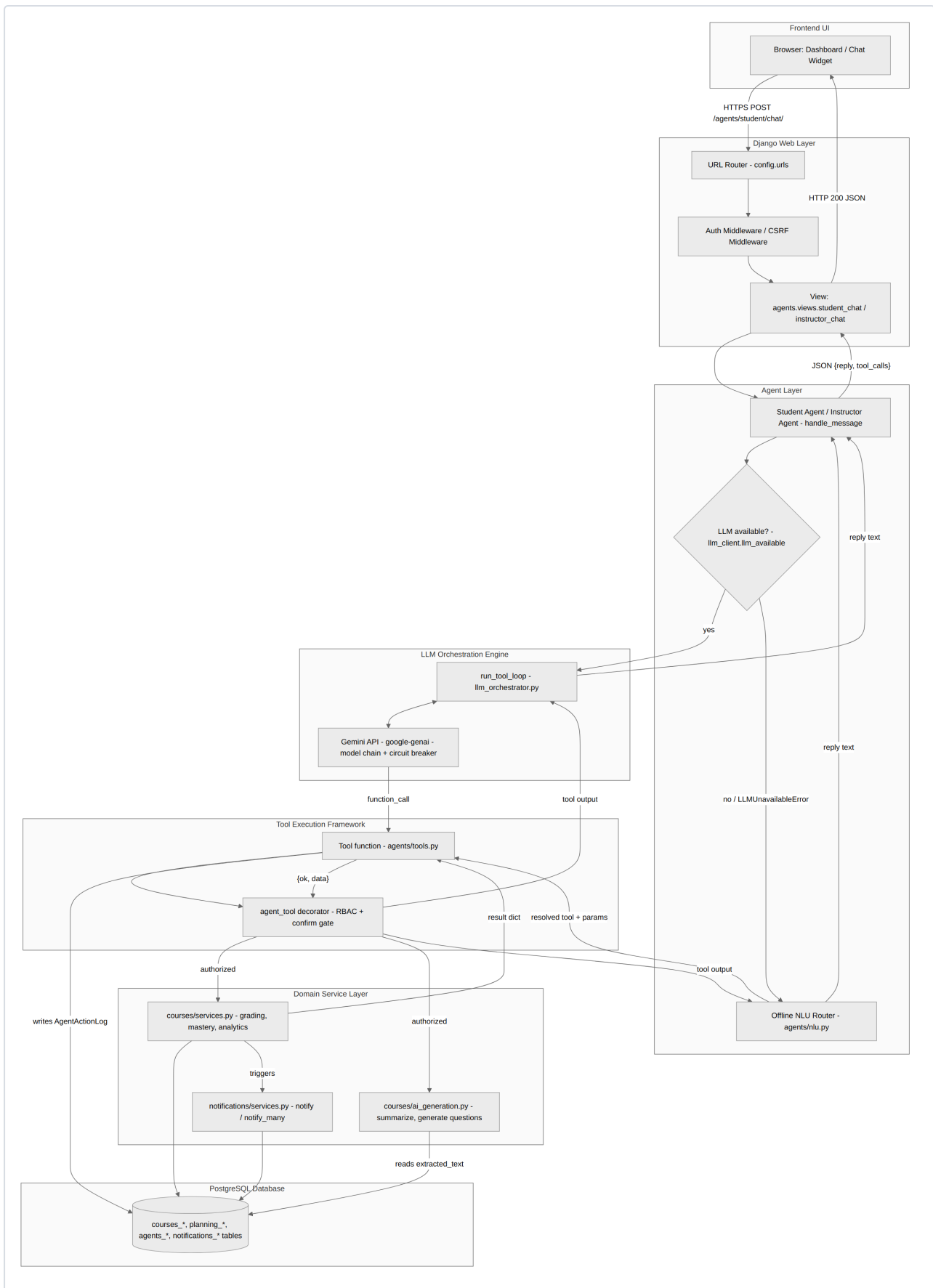
- **Two-layer enforcement.** RBAC is enforced twice for every agent action: (1) structurally, because the Student Agent's `TOOL_FUNCS` / `TOOL_SPECS` dictionary and the Instructor Agent's are disjoint — a student's conversation is never even offered an instructor tool to call, and vice versa; and (2) defensively, inside every individual tool function, which re-checks `user.is_student` / `user.is_instructor` and re-filters every database lookup by ownership or enrollment. A bug or bypass at layer (1) is still caught by layer (2).
- **Ownership, not identity, gates access.** Being an instructor is necessary but not sufficient to act on a given `Course`, `Quiz`, or `Material` — every relevant query additionally filters on `instructor=user` (or the corresponding chain, e.g., `attempt_quiz_course_instructor=user` for grading). The same pattern applies to students via `students=user` / enrollment checks.
- **Cross-boundary review is explicitly bridged, not opened.** `review_study_plan` is the one Instructor Agent tool that touches a resource (`StudyPlan`) with no direct `course` foreign key; it is authorized instead by checking that the plan's student is enrolled in *any* course the calling instructor owns (`plan.student.courses_enrolled.filter(instructor=user).exists()`), which is a narrower, purpose-built check rather than a blanket relaxation of the ownership rule.
- **The Chatbot has no write capability at all.** It is architecturally incapable of Create/Update/Delete/Execute-mutating actions: `chatbot/services.py` imports only read models (`TopicMastery`, `QuizAttempt`, `Course`) and never imports `agents.tools`. This is enforced by module boundaries, not by a runtime permission check, so it cannot be bypassed by any prompt content.

5. System Architecture Diagram

5.1 Layers

Layer	Technology	Responsibility
Frontend UI	Django server-rendered templates (<code>templates/</code>), a single shared stylesheet (<code>static/css/main.css</code>), and plain JavaScript modules (<code>chat.js</code> , <code>dashboard.js</code> , <code>theme.js</code>)	Renders dashboards, analytics pages, chat widgets, and forms; issues <code>fetch()</code> calls to JSON endpoints for chat and dashboard refresh; persists theme preference client-side. No SPA framework, HTMX, or Alpine.js is used.
Django Web Layer	Django 6.1 (<code>config</code> project; <code>accounts</code> , <code>courses</code> , <code>planning</code> , <code>notifications</code> apps' views/urls)	Routing, session authentication, CSRF enforcement, form validation, request/response lifecycle, template context assembly (including the <code>notifications</code> and <code>agents.context_processors.agent_result</code> context processors).
Agent Layer	<code>agents</code> app (<code>views.py</code> , <code>student_agent.py</code> , <code>instructor_agent.py</code> , <code>nlu.py</code> , <code>decorators.py</code> , <code>models.py</code>)	Receives chat/action HTTP requests, resolves the caller's role-appropriate agent, drives the tool-calling loop or the offline router, and persists every call to <code>AgentActionLog</code> .
LLM Orchestration Engine	<code>agents/llm_orchestrator.py</code> + <code>llm_client.py</code> (Google Gemini via <code>google-genai</code> SDK)	Converts the agent's JSON-Schema <code>TOOL_SPECS</code> into Gemini <code>FunctionDeclaration</code> s, drives the iterative generate → tool-call → tool-result loop (bounded to 4 iterations), and implements the model fallback chain, rate-limit circuit breaker, and error classification shared by every AI surface.
Tool Execution Framework	<code>agents/tools.py</code> , <code>agents/decorators.py</code> , <code>agents/exceptions.py</code>	The concrete, individually authorized, individually audited Python functions the model (or the offline router) may invoke; wraps every call with permission checks, <code>AgentActionLog</code> writes, uniform error shaping, and the destructive-action confirmation gate.
Domain Service Layer	<code>courses/services.py</code> , <code>courses/ai_generation.py</code> , <code>courses/text_extraction.py</code> , <code>notifications/services.py</code>	Business logic that outlives any single tool call: grading, mastery recomputation, recommendation computation, class-performance analytics, AI content generation with deterministic heuristic fallbacks, and notification dispatch.
PostgreSQL Database	PostgreSQL 16 (default) / SQLite (optional dev fallback via <code>DB_ENGINE=sqlite</code>)	System of record for all six apps' models; enforces foreign-key integrity, composite uniqueness constraints, and (in production) ACID transactional guarantees.

5.2 Request–Response Lifecycle Diagram



5.3 Provider-Agnostic Design Note

The tool-calling loop (`llm_orchestrator.py`) is deliberately decoupled from the transport (`llm_client.py`): every module that needs the model (`agents/llm_orchestrator.py` , `courses/ai_generation.py` , `chatbot/services.py`) calls the same `llm_client.generate` / `llm_client.generate_text` functions and catches the same `LLMUnavailableError` . The `TOOL_SPECS` dictionaries in `student_agent.py` / `instructor_agent.py` are plain JSON Schema (originally written for Anthropic's Messages API); on the Gemini transport they are passed through unchanged via `parameters_json_schema` , meaning a future provider swap requires changes only inside `llm_client.py` and `llm_orchestrator.py` 's `_to_gemini_tools` adapter — none of the 33 tool functions or their specs need to change.

6. Database Design & ERD

6.1 Reference Note

For the complete visual entity-relationship structure — Chen notation, with entities as rectangles, relationships as diamonds, and attributes as ovals, grouped by app boundary (accounts, courses structure, courses quizzes & assessments, materials & progress, topic mastery, agents & notifications, planning) — refer to Figures 6.1–6.5 on the following pages. The data dictionary below is the authoritative textual companion to those diagrams and reflects the actual Django model definitions and the LMS_Database_Schema.docx reference.

6.2 Data Dictionary

accounts_user (extends Django's **AbstractUser**)

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK, auto-increment	PK	Surrogate primary key.
username	varchar(150)	UNIQUE, NOT NULL	—	Login identifier.
password	varchar(128)	NOT NULL	—	Django-hashed password (PBKDF2/Argon2 per AUTH_PASSWORD_VALIDATORS).
email	varchar(254)	NULLABLE	—	Contact email.
first_name, last_name	varchar(150)	NULLABLE	—	Standard Django profile fields.
is_staff, is_active, is_superuser	boolean	NOT NULL, default False/True/False	—	Django Admin access flags.
date_joined, last_login	datetime	NOT NULL / NULLABLE	—	Account lifecycle timestamps.
role	varchar(20)	NOT NULL, choices: student / instructor / admin , default student	—	Custom field driving RBAC and which agent a user may call.

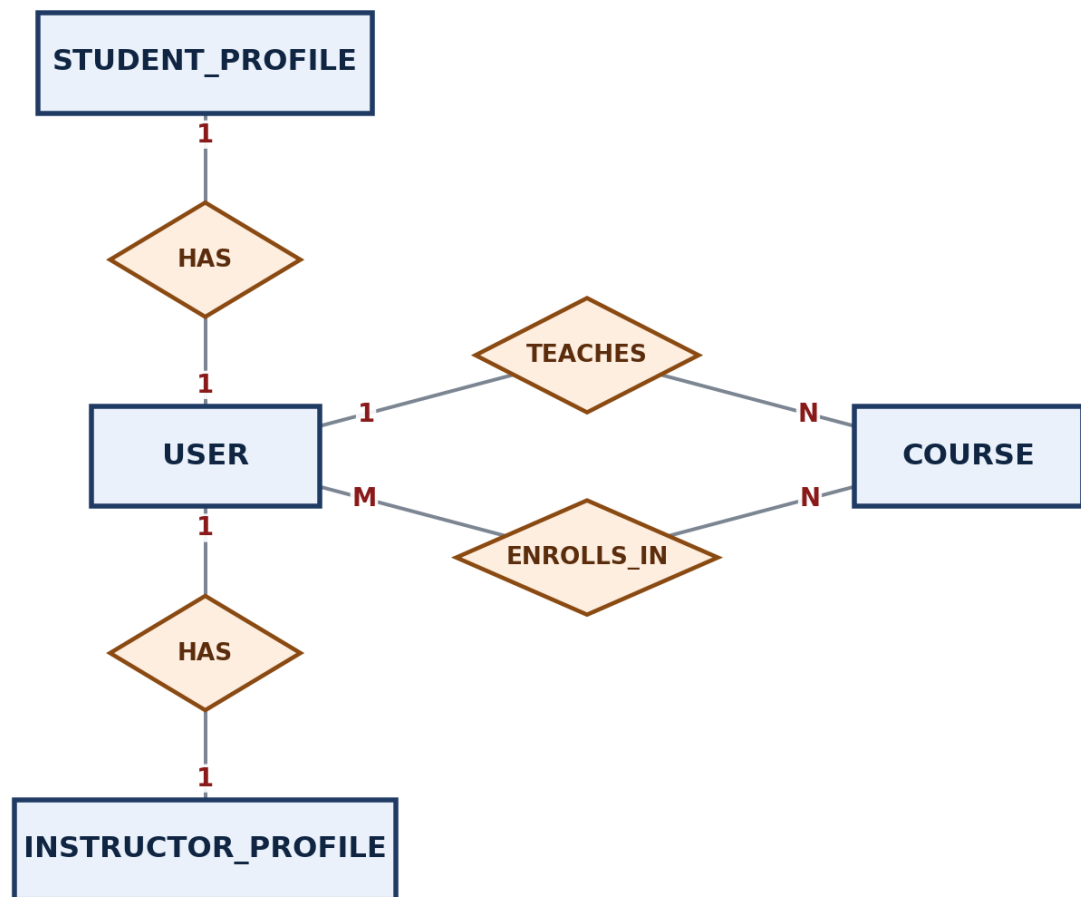
accounts_studentprofile

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
user_id	integer	UNIQUE, NOT NULL, ON DELETE CASCADE	FK → accounts_user.id (1-1)	Owning user; enforces exactly one profile per student.
bio	text	NULLABLE	—	Free-text biography.

6.1 Entity-Relationship Diagrams

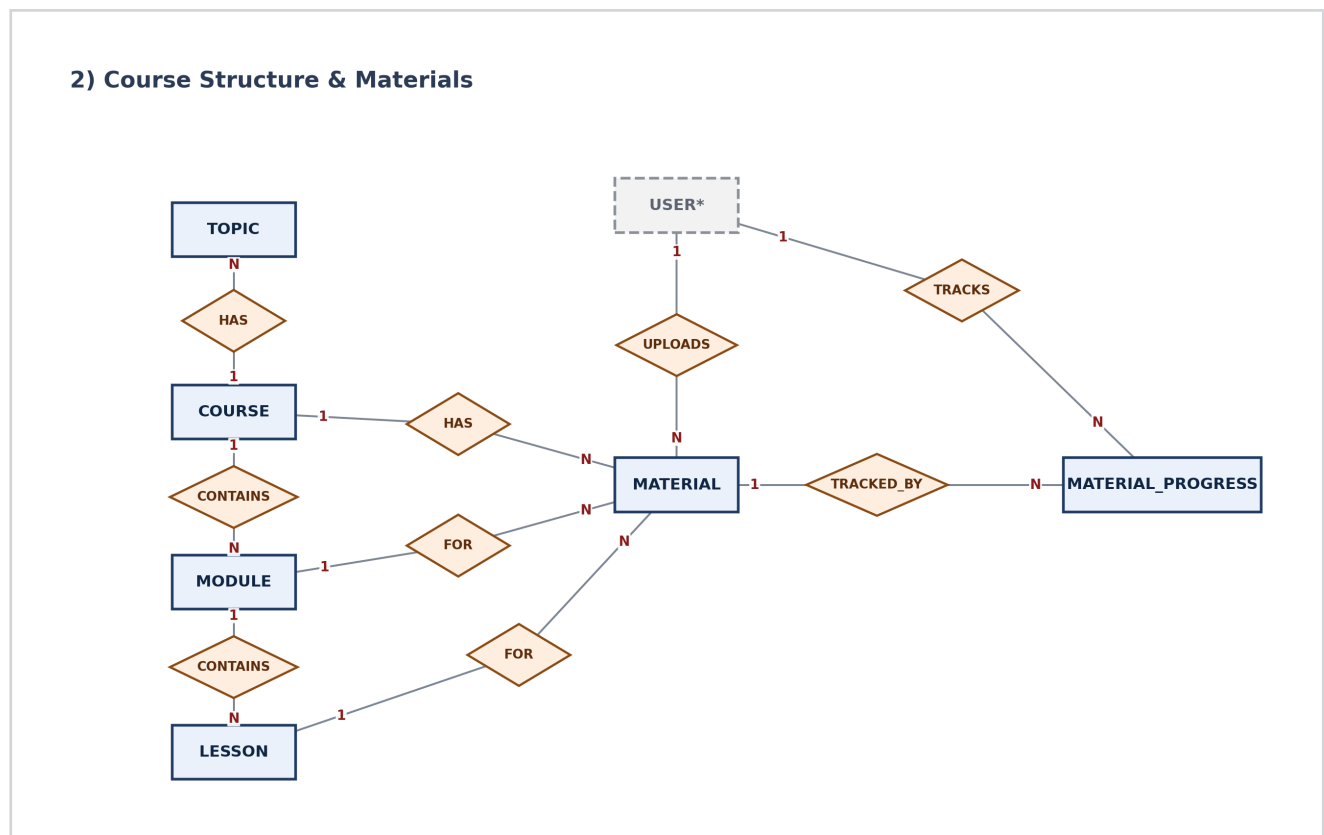
Figure 6.1 — Accounts & Enrollment (Chen ER Notation)

1) Accounts & Enrollment



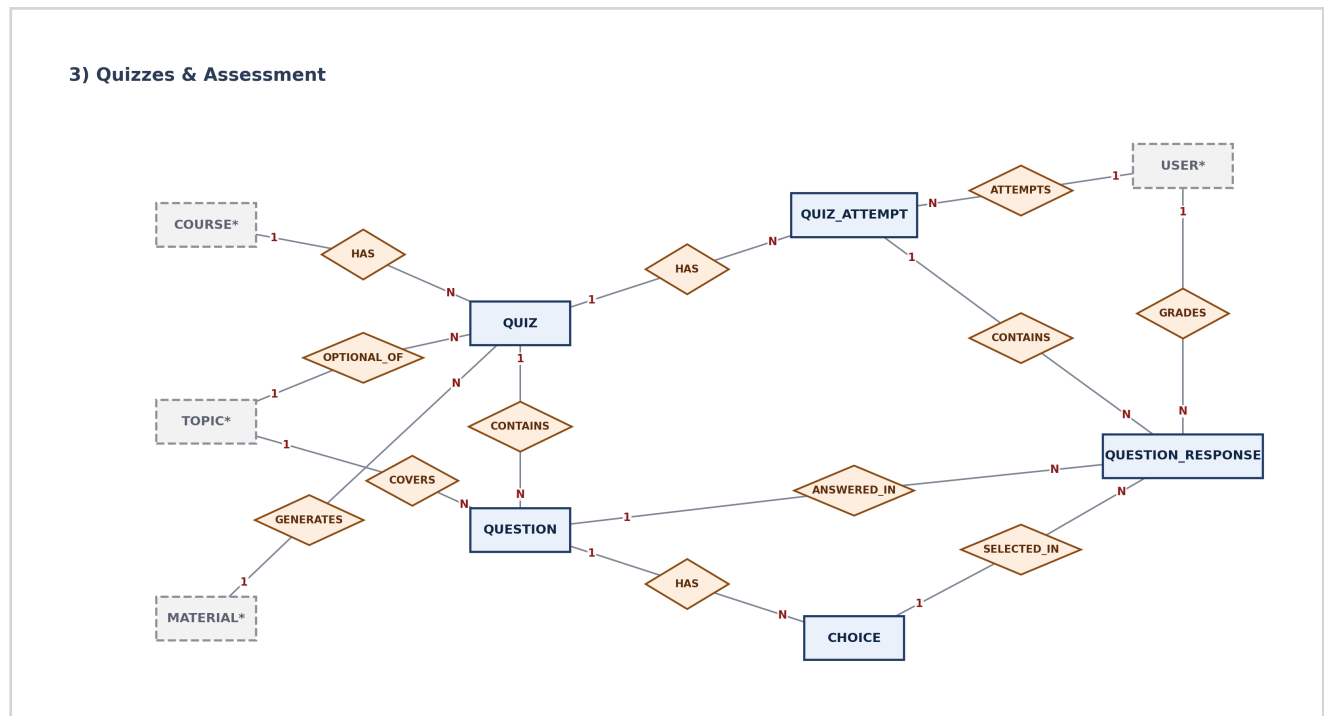
6.1 Entity-Relationship Diagrams (continued)

Figure 6.2 — Course Structure & Materials (Chen ER Notation)



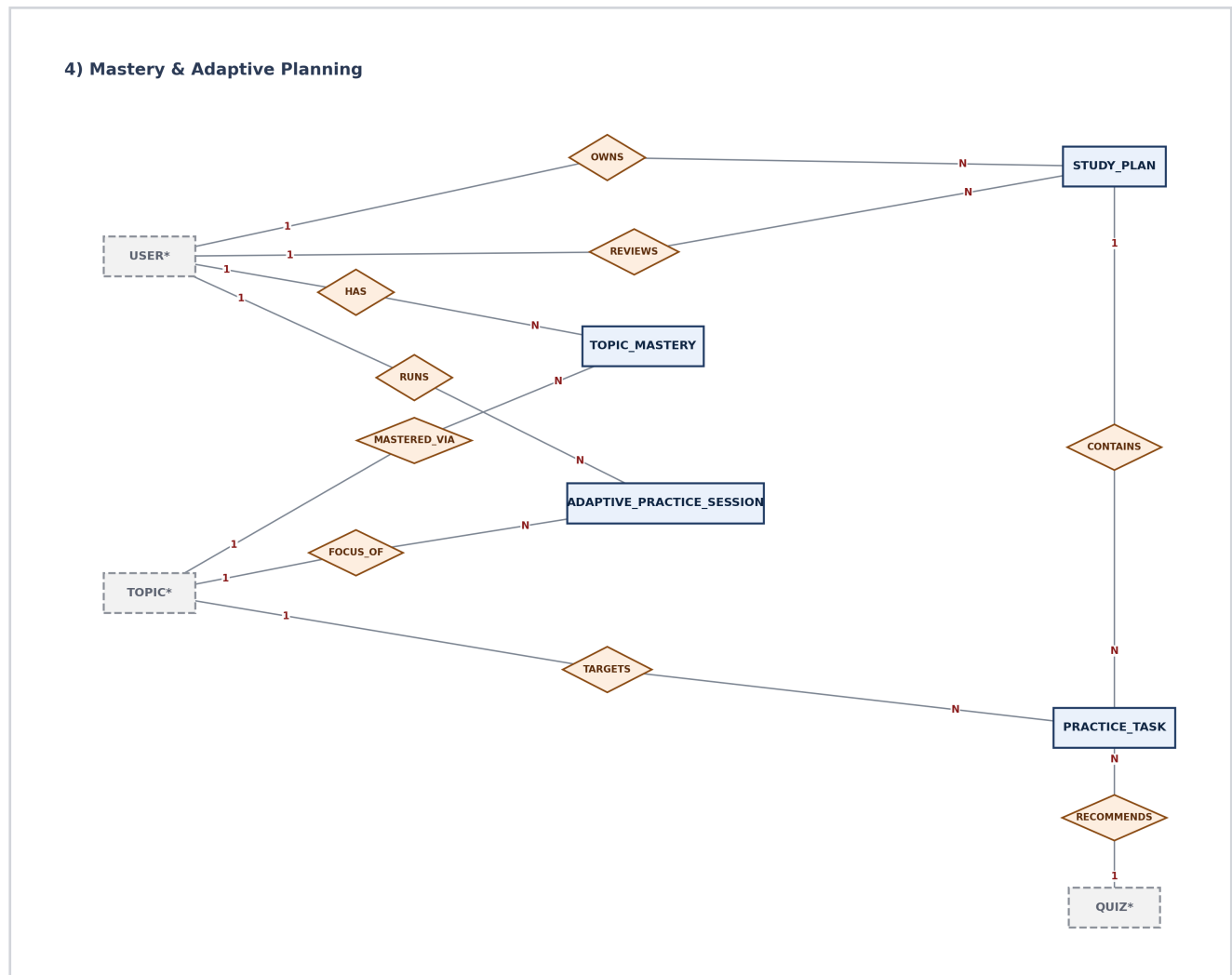
6.1 Entity-Relationship Diagrams (continued)

Figure 6.3 — Quizzes & Assessment (Chen ER Notation)



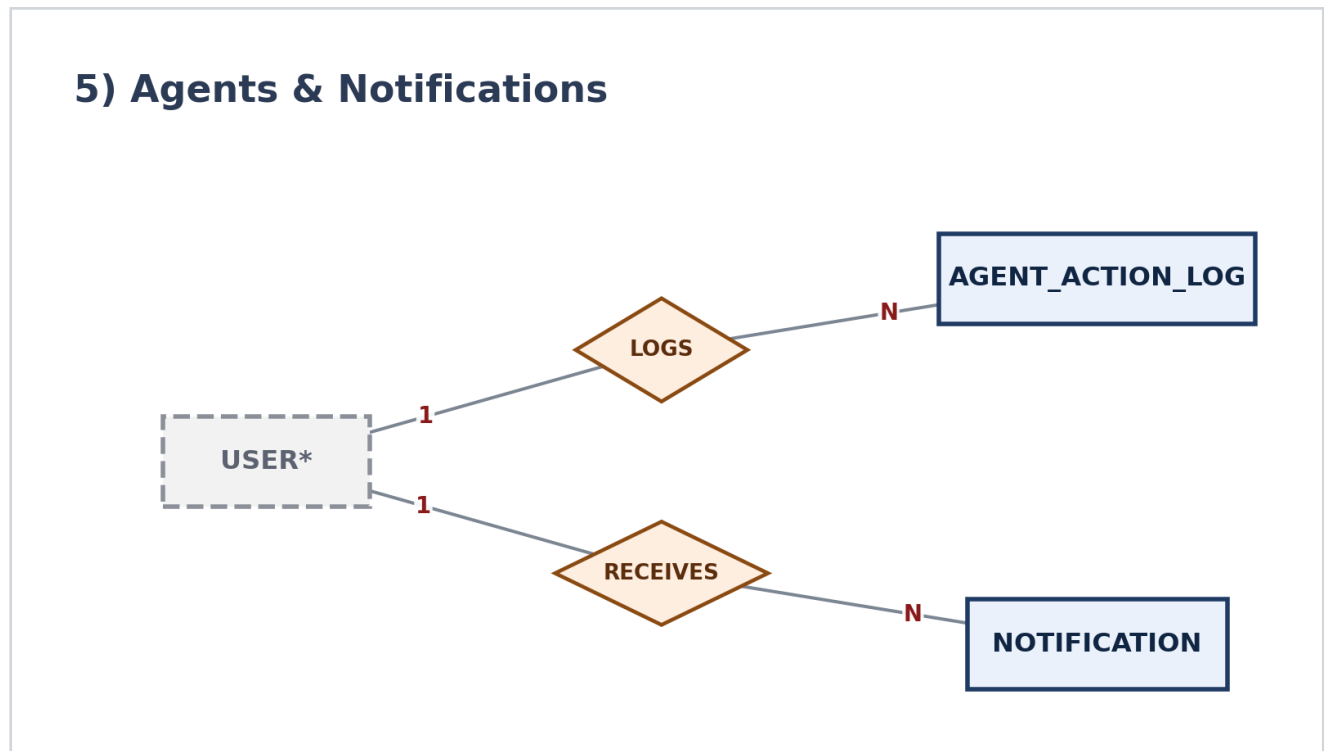
6.1 Entity-Relationship Diagrams (continued)

Figure 6.4 — Mastery & Adaptive Planning (Chen ER Notation)



6.1 Entity-Relationship Diagrams (continued)

Figure 6.5 — Agents & Notifications (Chen ER Notation)



Attribute	Type	Constraints	PK/FK	Description
level	varchar(50)	default "beginner"	—	Self-reported skill level.

accounts_instructorprofile

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
user_id	integer	UNIQUE, NOT NULL, ON DELETE CASCADE	FK → accounts_user.id (1-1)	Owning user.
bio	text	NULLABLE	—	Free-text biography.
department	varchar(100)	NULLABLE	—	Academic department affiliation.

courses_course

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
title	varchar(200)	NOT NULL	—	Course title.
description	text	NULLABLE	—	Course description.
instructor_id	integer	NOT NULL, ON DELETE (per project policy)	FK → accounts_user.id (N-1)	Exactly one instructor owns the course.
created_at	datetime	auto-set on create	—	Creation timestamp.
ai_summary	text	NULLABLE	—	Cached AI-generated course-level narrative summary.
ai_summary_generated_at	datetime	NULLABLE	—	Timestamp of the last AI summary generation.

courses_course_students (auto-generated M2M join table)

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
course_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_course.id	Enrolled course.
user_id	integer	NOT NULL, ON DELETE CASCADE	FK → accounts_user.id	Enrolled student.

Resolves the M-N relationship between **Course** and enrolled **User** (students); implicitly UNIQUE on (**course_id** , **user_id**) as generated by Django's **ManyToManyField** .

courses_module

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
course_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_course.id (N-1)	Parent course.
title	varchar(200)	NOT NULL	—	Module title.
order	integer	NOT NULL	—	Display sequence within the course.

courses_lesson

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
module_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_module.id (N-1)	Parent module.
title	varchar(200)	NOT NULL	—	Lesson title.
content	text	NULLABLE	—	Lesson body text.
order	integer	NOT NULL	—	Display sequence within the module.

courses_topic

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
course_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_course.id (N-1)	Parent course.
name	varchar(150)	NOT NULL, UNIQUE together with course_id	—	Topic label; the organizing tag for quizzes, mastery, study plans, and adaptive sessions.

courses_quiz

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
course_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_course.id (N-1)	Parent course.
topic_id	integer	NULLABLE, ON DELETE (nullable)	FK → courses_topic.id (N-1)	Optional topic tag.
title	varchar(200)	NOT NULL	—	Quiz title.

Attribute	Type	Constraints	PK/FK	Description
is_active	boolean	NOT NULL, default True	—	Whether the quiz currently accepts new attempts.
created_by_id	integer	NOT NULL	FK → <code>accounts_user.id</code> (N-1)	Authoring instructor.
created_via_agent	boolean	NOT NULL, default False	—	True if created by the Instructor Agent rather than the manual UI form.
generated_from_material_id	integer	NULLABLE, ON DELETE SET NULL	FK → <code>courses_material.id</code> (N-1)	The Material this quiz was AI-generated from, if any.
created_at	datetime	auto-set on create	—	Creation timestamp.
cancelled_at	datetime	NULLABLE	—	Set when <code>cancel_quiz</code> deactivates the quiz.

`courses_question`

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
quiz_id	integer	NOT NULL, ON DELETE CASCADE	FK → <code>courses_quiz.id</code> (N-1)	Parent quiz.
topic_id	integer	NOT NULL	FK → <code>courses_topic.id</code> (N-1)	Topic this question drills.
text	text	NOT NULL	—	Question prompt.
question_type	varchar(10)	NOT NULL, choices: <code>mcq</code> / <code>essay</code>	—	Determines auto-grading vs. manual grading path.
order	integer	NOT NULL	—	Display sequence within the quiz.
generated_by_ai	boolean	NOT NULL, default False	—	Provenance flag.

`courses_choice`

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
question_id	integer	NOT NULL, ON DELETE CASCADE	FK → <code>courses_question.id</code> (N-1)	Parent MCQ question.
text	varchar(300)	NOT NULL	—	Choice text.
is_correct	boolean	NOT NULL, default False	—	Marks the correct choice; exactly one per question is enforced at the application layer.

courses_quizattempt

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
quiz_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_quiz.id (N-1)	Quiz being attempted.
student_id	integer	NOT NULL, ON DELETE CASCADE	FK → accounts_user.id (N-1)	Attempting student.
started_at	datetime	auto-set on create	—	Attempt start time.
completed_at	datetime	NULLABLE	—	Set on submission.
score	float	NULLABLE, 0–100	—	Aggregate percentage score, recomputed whenever a constituent response changes.
status	varchar (20)	NOT NULL, choices: in_progress / pending_review / graded	—	Grading lifecycle state.

courses_questionresponse

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
attempt_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_quizattempt.id (N-1)	Parent attempt.
question_id	integer	NOT NULL	FK → courses_question.id (N-1)	Question being answered.
selected_choice_id	integer	NULLABLE, ON DELETE (nullable)	FK → courses_choice.id (N-1)	For MCQ answers.
answer_text	text	NULLABLE	—	For essay/short-answer answers.
is_correct	boolean	NULLABLE	—	Auto-graded outcome for MCQ.
needs_review	boolean	NOT NULL, default False	—	True for essay responses awaiting manual grading.
manual_score	float	NULLABLE, 0–1	—	Instructor-assigned score fraction.
graded_by_id	integer	NULLABLE, ON DELETE SET NULL	FK → accounts_user.id (N-1)	Grading instructor.
feedback	varchar(500)	NULLABLE	—	Instructor free-text feedback.
graded_at	datetime	NULLABLE	—	Grading timestamp.

courses_material

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
course_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_course.id (N-1)	Parent course.
module_id	integer	NULLABLE	FK → courses_module.id (N-1)	Optional module link.
lesson_id	integer	NULLABLE	FK → courses_lesson.id (N-1)	Optional lesson link.
title	varchar(200)	NOT NULL	—	Display title.
file	filepath	NOT NULL	—	Stored file path (MEDIA_ROOT -relative).
kind	varchar(10)	NOT NULL, choices: pdf / docx / pptx / txt / video / other	—	Detected file kind, from extension.
uploaded_by_id	integer	NOT NULL	FK → accounts_user.id (N-1)	Uploading instructor.
extracted_text	text	NULLABLE, ≤40,000 chars	—	Plain text extracted for AI grounding.
extraction_status	varchar(15)	NOT NULL, values: done / unsupported / failed / n/a	—	Outcome of text extraction.
ai_summary	text	NULLABLE	—	Instructor-generated, student-visible cached summary.
ai_summary_generated_at	datetime	NULLABLE	—	Timestamp of last summary generation.
created_at	datetime	auto-set on create	—	Upload timestamp.

courses_materialprogress

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
student_id	integer	NOT NULL, ON DELETE CASCADE	FK → accounts_user.id (N-1)	Owning student.
material_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_material.id (N-1)	Tracked material.
is_done	boolean	NOT NULL, default False	—	Completion flag.
done_at	datetime	NULLABLE	—	Completion timestamp.

Attribute	Type	Constraints	PK/FK	Description
—	—	UNIQUE(student_id, material_id)	—	At most one progress row per student/material pair.

courses_topicmastery

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
student_id	integer	NOT NULL, ON DELETE CASCADE	FK → accounts_user.id (N-1)	Owning student.
topic_id	integer	NOT NULL, ON DELETE CASCADE	FK → courses_topic.id (N-1)	Tracked topic.
mastery_percent	float	NOT NULL, 0–100	—	Rolling competency score, recomputed on every graded attempt touching this topic.
attempts_count	integer	NOT NULL, default 0	—	Number of attempts that have contributed to this score.
updated_at	datetime	auto-updated	—	Last recomputation timestamp.
—	—	UNIQUE(student_id, topic_id)	—	At most one mastery row per student/topic pair.

planning_studyplan

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
student_id	integer	NOT NULL, ON DELETE CASCADE	FK → accounts_user.id (N-1)	Owning student.
title	varchar (200)	NOT NULL	—	Plan title.
summary	text	NULLABLE	—	Structured, multi-section AI-authored plan narrative.
status	varchar (20)	NOT NULL, choices: pending / approved / rejected , default pending	—	Instructor review state.
reviewed_by_id	integer	NULLABLE, ON DELETE SET NULL, limit_choices_to={"role": "instructor"}	FK → accounts_user.id (N-1)	Reviewing instructor.
review_note	text	NULLABLE	—	Reviewer's free-text note.
created_via_agent	boolean	NOT NULL, default True	—	True for every current plan (Student Agent is the only creation path).

Attribute	Type	Constraints	PK/FK	Description
created_at	datetime	auto-set on create	—	Creation timestamp.
reviewed_at	datetime	NULLABLE	—	Review timestamp.

planning_practicetask

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
plan_id	integer	NOT NULL, ON DELETE CASCADE	FK → <i>planning_studyplan.id</i> (N-1)	Parent plan.
topic_id	integer	NOT NULL, ON DELETE CASCADE	FK → <i>courses_topic.id</i> (N-1)	Target topic.
recommended_quiz_id	integer	NULLABLE, ON DELETE SET NULL	FK → <i>courses_quiz.id</i> (N-1)	Optional linked quiz.
description	varchar(300)	NOT NULL	—	Task description.
priority	varchar(10)	NOT NULL, choices: <i>high</i> / <i>medium</i> / <i>low</i> , default <i>medium</i>	—	Priority tier, derived from mastery gap at creation.
is_done	boolean	NOT NULL, default False	—	Completion flag, independent of plan status.
created_at	datetime	auto-set on create	—	Creation timestamp.

planning_adaptivepracticesession

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
student_id	integer	NOT NULL, ON DELETE CASCADE	FK → <i>accounts_user.id</i> (N-1)	Owning student.
topic_id	integer	NOT NULL, ON DELETE CASCADE	FK → <i>courses_topic.id</i> (N-1)	Practiced topic.
context_text	text	NULLABLE	—	Cached grounding text (relevant materials or synthesized primer) reused for every question in the session.
difficulty	varchar(10)	NOT NULL, choices: <i>easy</i> / <i>medium</i> / <i>hard</i> , default <i>medium</i>	—	Current adaptive difficulty.
streak_correct, streak_wrong	integer	NOT NULL, default 0	—	Consecutive-answer counters driving difficulty transitions.

Attribute	Type	Constraints	PK/FK	Description
questions_asked, questions_correct	integer	NOT NULL, default 0	—	Running session totals.
current_question_text	text	NULLABLE	—	The question currently awaiting an answer.
current_choices	JSON	default <code>[]</code> , shape <code>[{"text": str, "is_correct": bool}, ...]</code>	—	Choices for the current question, including the answer key server-side only.
is_active	boolean	NOT NULL, default True	—	At most one active session per student, enforced in the tool layer.
created_at, updated_at	datetime	auto-set / auto-updated	—	Lifecycle timestamps.

agents_agentactionlog

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
user_id	integer	NOT NULL, ON DELETE CASCADE	FK → <code>accounts_user.id</code> (N-1)	Calling user (student or instructor).
agent	varchar(30)	NOT NULL, choices: <code>student_agent</code> / <code>instructor_agent</code>	—	Which agent handled the call.
tool_name	varchar(100)	NOT NULL	—	Function name of the invoked tool.
params	JSON	default <code>{}</code> , JSON-safe	—	Sanitized copy of the input arguments.
result	JSON	default <code>{}</code> , JSON-safe	—	Sanitized copy of the output (or the error / confirmation prompt).
status	varchar(20)	NOT NULL, choices: <code>success</code> / <code>denied</code> / <code>error</code> / <code>pending_confirm</code>	—	Outcome of the call.
timestamp	datetime	auto-set on create, default ordering - <code>timestamp</code>	—	When the call occurred.

notifications_notification

Attribute	Type	Constraints	PK/FK	Description
id	integer	PK	PK	Surrogate key.
user_id	integer	NOT NULL, ON DELETE CASCADE	FK → <code>accounts_user.id</code> (N-1)	Recipient.
message	varchar(300)	NOT NULL	—	Notification text.
is_read	boolean	NOT NULL, default False	—	Read/unread flag.

Attribute	Type	Constraints	PK/FK	Description
created_at	datetime	auto-set on create	—	Creation timestamp.

chatbot app — no persisted tables

The **chatbot** app's `models.py` is empty. The Chatbot is deliberately stateless: it reads existing data from `courses` models (`TopicMastery` , `QuizAttempt` , `Course`) at request time and returns a reply without writing anything to the database — a structural (not merely policy-level) guarantee that it cannot mutate state or contribute an audit trail entry.

6.3 Relationship Summary

Relationship	Cardinality	Notes
User ↔ StudentProfile	1–1	<code>related_name="student_profile"</code> .
User ↔ InstructorProfile	1–1	<code>related_name="instructor_profile"</code> .
User → Course (as instructor)	1–N	One instructor per course.
User ↔ Course (as enrolled student)	N–N	Via <code>courses_course_students</code> .
Course → Module → Lesson	1–N, 1–N	Two-level content hierarchy.
Course → Topic	1–N	Unique topic name per course.
Course → Quiz; Topic → Quiz	1–N; 1–N (optional)	A quiz always belongs to a course; a topic tag is optional.
Quiz → Question → Choice	1–N, 1–N	Choices only meaningful for <code>question_type = mcq</code> .
Quiz → QuizAttempt; User → QuizAttempt	1–N; 1–N	An attempt belongs to exactly one quiz and one student.
QuizAttempt → QuestionResponse	1–N	Each response links to one question and optionally one choice.
Course → Material	1–N	Material may optionally link to one Module and/or one Lesson.
User ↔ Material (via MaterialProgress)	N–N	Join table carries <code>is_done</code> / <code>done_at</code> .
User ↔ Topic (via TopicMastery)	N–N	Join table carries <code>mastery_percent</code> / <code>attempts_count</code> .
User → StudyPlan → PracticeTask	1–N, 1–N	A plan belongs to one student; tasks belong to one plan.
PracticeTask → Topic; PracticeTask → Quiz (optional)	N–1; N–1 (optional)	Task always targets a topic, optionally references a recommended quiz.
User → AdaptivePracticeSession; Topic → AdaptivePracticeSession	1–N; 1–N	Session belongs to one student, targets one topic.
User → AgentActionLog	1–N	Every tool call, regardless of outcome.
User → Notification	1–N	In-app notifications.

6.4 3NF & Normalization Notes

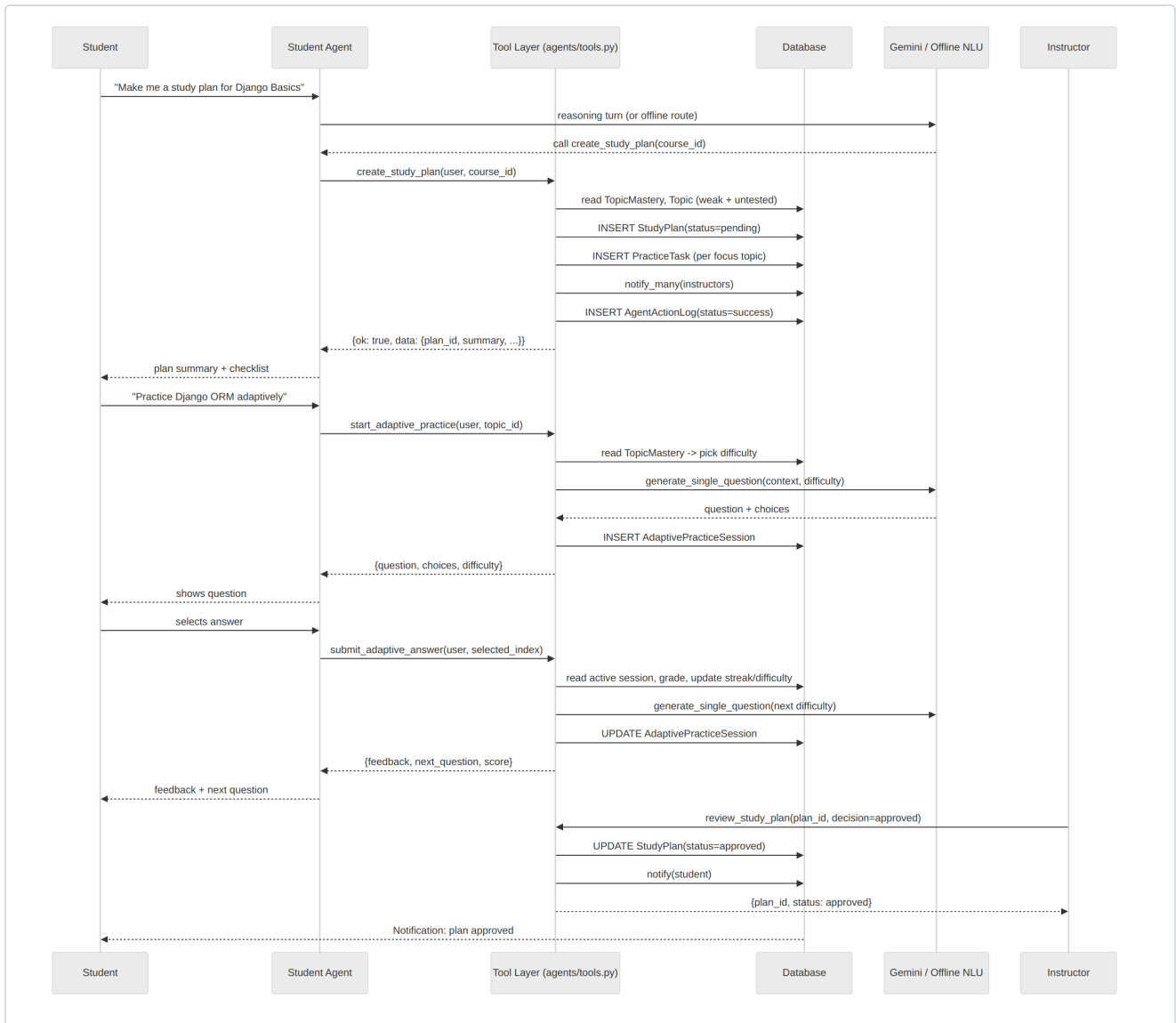
- **1NF (atomic values):** every column holds a single atomic value. The two JSON columns (`AgentActionLog.params / result` , `AdaptivePracticeSession.current_choices`) are the schema's sole deliberate exceptions, and both are justified: `AgentActionLog` 's columns must accept an arbitrary, tool-specific parameter/result shape by design (an audit log cannot have a fixed column per possible tool argument without an unbounded, ever-growing table), and `current_choices` stores transient, single-purpose session state that is never queried or filtered on — only read/written wholesale by the one tool pair (`start_adaptive_practice / submit_adaptive_answer`) that owns it. Neither is used as a substitute for a proper relational table where relational structure was actually needed.
- **2NF (no partial dependency on a composite key):** every table in the schema uses a single-column surrogate integer primary key (`id`), so partial-key dependency cannot arise by construction; the schema instead expresses natural composite constraints as **unique constraints** layered on top of the surrogate key (`MaterialProgress` on `student_id + material_id` , `TopicMastery` on `student_id + topic_id` , `Topic` on `name + course_id`) rather than as composite primary keys, which keeps every foreign-key reference elsewhere in the schema a simple single-column reference.
- **3NF (no transitive dependency):** no non-key column depends on another non-key column rather than directly on its own table's primary key. The clearest illustration is `courses_quizattempt.score` : it is *not* stored as a formula-dependent duplicate of the sum of its `QuestionResponse` rows kept in sync manually — it is a materialized aggregate recomputed by the single service function `courses/services.py:_recompute_score` every time a dependent `QuestionResponse` changes, which is the standard, intentional exception RDBMS design makes for materialized aggregates (identical in spirit to `TopicMastery.mastery_percent`): the "source of truth" is the child rows, the parent column is a maintained cache of a computation over them, not an independently-editable fact that could drift out of sync with a differently-derived value.
- **Many-to-many resolution via join tables:** the `Course` ↔ `User` (enrolled students) relationship is resolved by Django's auto-generated `courses_course_students` table rather than a repeating-group column (e.g., a comma-separated list of student IDs on `Course`), and likewise `User` ↔ `Material` progress and `User` ↔ `Topic` mastery are resolved by explicit join tables (`MaterialProgress` , `TopicMastery`) that additionally carry relationship-specific attributes (`is_done` , `mastery_percent`) — the textbook justification for promoting an M2M join table to a first-class model rather than leaving it as a bare Django `ManyToManyField` .
- **No repeating groups:** `Question.question_type` (`mcq / essay`) branches storage into two genuinely different shapes (a set of `Choice` rows vs. free-text `answer_text`) rather than cramming both into optional columns on one wide row family; `Choice` exists as its own table precisely because an MCQ's option count is variable, which a fixed set of `choice_1 ... choice_n` columns on `Question` would have violated 1NF/3NF to accommodate.
- **Referential-integrity-driven on_delete choices reinforce normalization intent:** cascading deletes (`CASCADE`) are used only where the child row is meaningless without its parent (a `Choice` without its `Question` , a `PracticeTask` without its `StudyPlan`); nullifying deletes (`SET_NULL`) are used specifically where the child row remains independently meaningful after the parent's removal (a `Quiz.generated_from_material` after the source `Material` is deleted — see FR-2.2.11 — or a `QuestionResponse.graded_by` after a grading instructor's account is removed), preventing normalization-correct but operationally destructive cascade chains.

7. User Journey Flows

7.1 Student Adaptive Practice & Study Plan Journey

Step-by-step:

1. The student opens their dashboard and sees `get_learning_dashboard` output: overall progress, quiz average, and their weakest topic highlighted by name.
2. The student asks the Student Agent (in chat, in English or Egyptian Arabic) to build a study plan for a course they're enrolled in.
3. The agent calls `create_study_plan(course_id)` : it reads the student's `TopicMastery` rows for that course, ranks the three weakest by `mastery_percent` , identifies up to three never-attempted topics, assigns each a priority, writes a structured multi-section summary, creates the `StudyPlan` (`status = pending`), seeds one `PracticeTask` per focus topic, and notifies the course's instructor.
4. The student reviews the seeded checklist and picks the top-priority task; they ask to practice that topic adaptively.
5. The agent calls `start_adaptive_practice(topic_id)` : it deactivates any previous session, reads the student's current mastery on that topic to pick a starting difficulty, generates one AI question grounded in relevant course materials (or a synthesized primer), and persists an `AdaptivePracticeSession` .
6. The student answers; the agent calls `submit_adaptive_answer(selected_index)` , which grades the choice, adjusts the running streaks and difficulty (harder after two correct in a row, easier after one wrong), and immediately returns the next question at the new difficulty.
7. Steps 5–6 repeat for several questions; the student ends the loop by asking to stop, triggering `end_adaptive_practice` , which returns a session summary (questions asked/correct, final difficulty).
8. Separately, the course's instructor sees the pending-plan notification, opens the full-screen Study Plan page, and calls `review_study_plan(plan_id, decision="approved", note=...)` .
9. The student receives an in-app `Notification` that their plan was approved, with the instructor's note if one was given, and continues working through the now-approved checklist, marking tasks done via `mark_task_done` as they complete them.

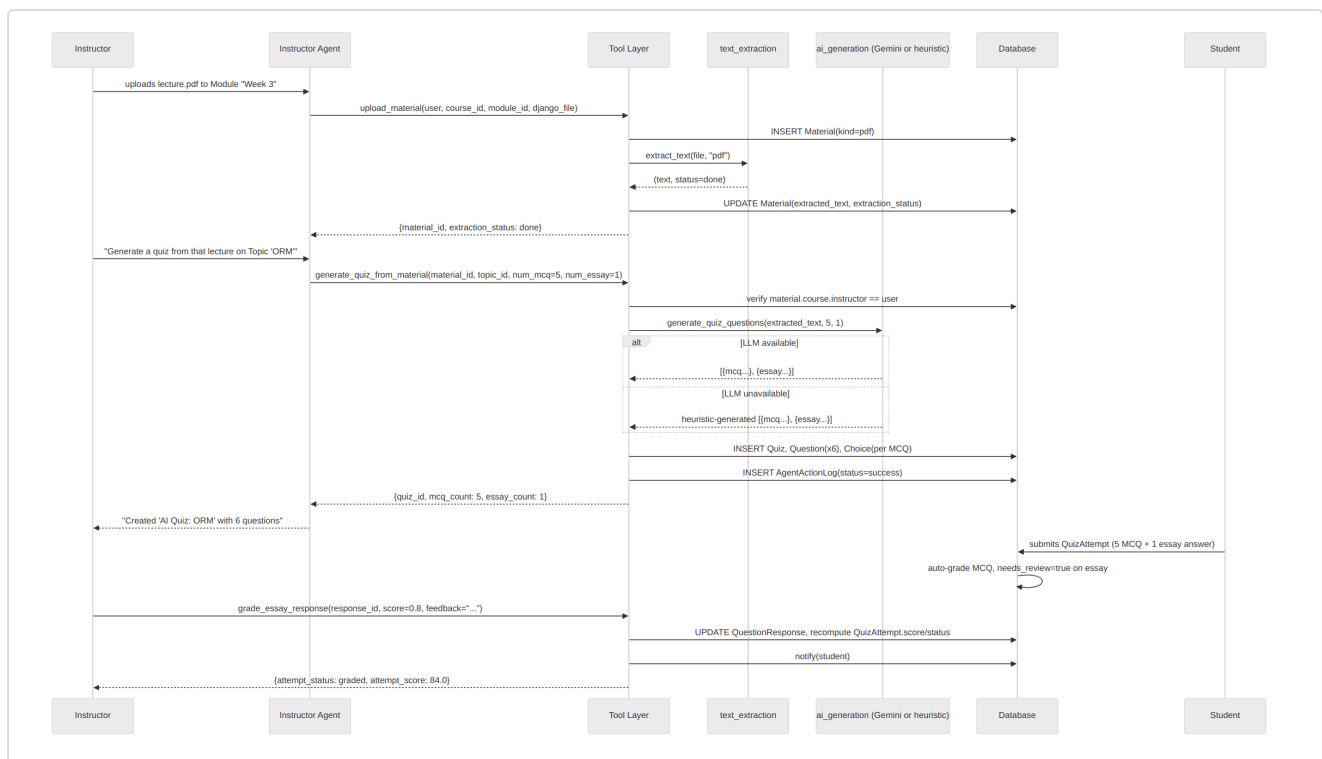


7.2 Instructor AI-Assisted Quiz Generation from Course Materials

Step-by-step:

1. The instructor uploads a lecture PDF to a course module via the Instructor Agent (`upload_material` , backed by a `<form>` POST that saves the file and calls the tool with the saved `Django File` object).
2. The tool creates the `Material` row, classifies its `kind` from the file extension, and immediately runs `text_extraction.extract_text` , writing `extracted_text` and `extraction_status` (`done` , `unsupported` , or `failed`) back onto the row.
3. The instructor asks the agent to generate a quiz from that material for a specific topic.
4. The agent calls `generate_quiz_from_material(material_id, topic_id, num_mcq, num_essay)` ; it validates the material belongs to the instructor's own course and that its extraction status is `done` , then calls `ai_generation.generate_quiz_questions(extracted_text, num_mcq, num_essay)` .

5. `ai_generation` calls the resilient LLM client; if the model is unavailable for any reason, it silently falls back to a deterministic heuristic generator (built from the material's own sentences) rather than raising, guaranteeing the tool never surfaces "Internal tool error" to the instructor for this reason.
6. The tool builds the `Quiz`, one `Question` per generated item (MCQ questions additionally get their `Choice` rows), links `generated_from_material`, and returns the quiz ID plus MCQ/essay counts.
7. The instructor reviews the quiz page; if they want one more manually-authored question, they call `add_question_to_quiz` directly.
8. A student later takes the quiz; MCQ answers are auto-graded on submission (`courses/services.py:submit_attempt`), while essay answers are flagged `needs_review = true`.
9. The instructor sees the pending-essay count on `get_course_analytics`, opens the response, and calls `grade_essay_response(response_id, score, feedback)`, which recomputes the attempt's aggregate score/status and notifies the student.



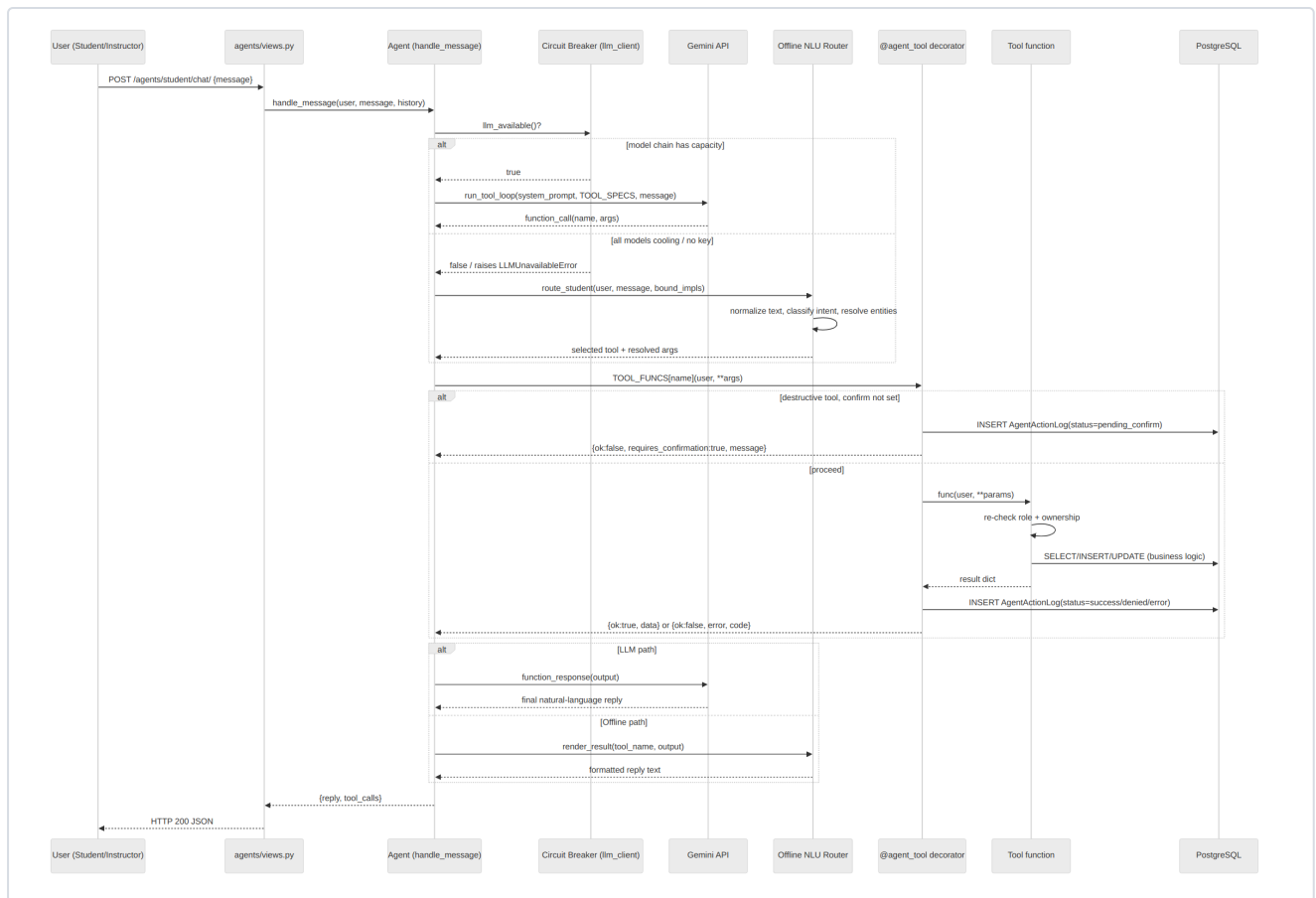
7.3 End-to-End AI Agent Tool Execution Workflow

Step-by-step (Intent Parsing → NLU → Tool Call → DB Mutation → Result Formatting):

1. **Intent parsing (input):** the user's raw chat message (`message`) arrives at `agents/views.py`'s `student_chat` or `instructor_chat` endpoint via an authenticated, CSRF-protected POST, and is handed to `student_agent.handle_message(user, message, history)` / the instructor equivalent.
2. **Routing decision:** `handle_message` checks `llm_client.llm_available()` — true only if the SDK is installed, a `GEMINI_API_KEY` is set, `AI_OFFLINE_MODE` is not forced, and at least one model in the chain is not currently on cooldown.
- 3a. **LLM path (NLU by the model itself):** if available, `run_tool_loop` sends the system prompt, the full `TOOL_SPECS` JSON-Schema catalogue, and the message to Gemini. The model

performs its own intent classification and argument extraction, returning either free text or one or more `function_call` parts. 3b. **Offline path (rule-based NLU):** if unavailable — or if `run_tool_loop` raises `LLMUnavailableError` mid-conversation — `agents/nlu.py`'s `route_student` / `route_instructor` normalizes the text (case-folding, Arabic diacritic/letter-form normalization), matches it against per-tool English/Arabic keyword patterns to classify the intent, then resolves entities (course/topic/material/student/quiz/plan by name or number, defaulting to "the only one you have" when unambiguous) directly from the sentence.

3. **Tool call:** whichever path won, the resolved tool name and arguments are passed to `call_tool(user, name, **params)`, which dispatches to the bound function in `TOOL_FUNCS` — the exact same function object regardless of which NLU path selected it.
4. **Authorization + destructive-action gate:** the `@agent_tool` decorator wrapping that function first checks, for destructive tools, whether `confirm=true` was supplied; if not, it logs a `pending_confirm` audit row and returns a confirmation prompt without touching any other table.
5. **DB mutation (business logic):** the tool function itself re-validates the caller's role and ownership, then performs its reads/writes, frequently delegating multi-row consistency operations (grading, mastery recomputation) to `courses/services.py`, and any notification dispatch to `notifications/services.py`.
6. **Audit write:** regardless of success or failure, the decorator writes exactly one `AgentActionLog` row capturing the sanitized params and result.
7. **Result formatting:** on the LLM path, the tool's `{ok, data}` (or `{ok: false, error, code}`) dict is fed back to the model as a `function_response` part, and the model composes the final natural-language reply for the user. On the offline path, `agents/nlu.py:render_result` deterministically formats the same dict into a readable sentence — no LLM call is needed to produce the reply.
8. **Response delivery:** `handle_message` returns `{"reply": <text>, "tool_calls": [...]}`, which the view serializes as JSON back to the chat widget's `fetch()` call.



8. AI & Agent Design

8.1 Agent Architecture

`agents/student_agent.py` defines `AGENT_NAME = "student_agent"`, a `TOOL_FUNCS` dict mapping 16 tool names to bound functions in `agents/tools.py`, a matching `TOOL_SPECS` JSON-Schema list, a `SYSTEM_PROMPT` instructing the model to act only through tools, to relay (and wait on) destructive-action confirmations, to ask (or infer) a summary style before calling `summarize_material`, to prefer `ask_about_material` over `explain_topic` when the student is reading a specific lecture, and to prefer `generate_practice_quiz` for a one-off "quiz me" request versus `start_adaptive_practice` / `submit_adaptive_answer` for progressively harder practice. `handle_message(user, message, history)` is the single entry point: it checks `llm_available()`, attempts `run_tool_loop` if so, catches `LLMUnavailableError`, and falls back to `_fallback_router`, which calls `agents.nlu.route_student`.

`agents/instructor_agent.py` mirrors this structure with `AGENT_NAME = "instructor_agent"`, 17 tools, and a system prompt with equivalent guidance — notably instructing the model to default to a mixed MCQ/essay split (e.g., `num_mcq=5`, `num_essay=0`) unless the instructor explicitly asks for essay-only, and to prefer `generate_quiz_for_topic` over one-by-one `add_question_to_quiz` calls whenever the instructor's request is a general "make a quiz on topic X" rather than referencing one specific already-open material.

`agents/llm_orchestrator.py` is a thin, provider-specific adapter: `_to_gemini_tools` converts the agent's Anthropic-style JSON-Schema `TOOL_SPECS` into Gemini `types.FunctionDeclaration`s via `parameters_json_schema` (no per-tool rewriting needed), and `run_tool_loop` drives up to 4 iterations of generate → execute any returned `function_calls` → feed `function_responses` back → generate again, returning as soon as the model responds with text and no further calls, or after 4 iterations with a "reached my step limit" message, or — if tool calls ran but the final text came back empty — by rendering the tool calls' own results via `nlu.render_result` rather than surfacing a blank reply.

Historical note on provider migration: the module's own docstring records that the project was originally built on Anthropic's Claude API and was switched to Google's Gemini API (`google-genai` SDK) after the project's Anthropic credits were exhausted; this SRS reflects the current, as-shipped Gemini-based implementation, in which `agents/llm_orchestrator.py`, `courses/ai_generation.py`, and `chatbot/services.py` are the only three modules aware of which provider is in use — every agent tool remains provider-agnostic.

8.2 Tool Registry

All tools are decorated with `@agent_tool(agent_name, destructive=False)` (`agents/decorators.py`), which enforces the audit-logging, uniform-error-shaping, and destructive-confirmation behavior described in §3.2 and §8.3 for every entry below. Every "Output" column describes the payload nested under `data` in a successful `{"ok": true, "data": {...}}` response.

Student Agent Tools (16)

Tool Name	Description	Input Arguments	Output	DB Side-Effects
<code>get_performance</code>	Mastery percentage per topic across all enrolled courses.	<i>(none)</i>	<code>{topics: [{topic, course, mastery_percent, attempts}]}</code>	Read-only.
<code>create_study_plan</code>	Create a pending study plan from the student's weakest/untested topics in one course.	<code>course_id: int</code>	<code>{plan_id, status, summary, weak_topic_ids, seeded_task_count}</code>	INSERT <code>StudyPlan</code> ; INSERT <code>PracticeTask</code> ($\times N$); notify instructor(s).
<code>create_practice_task</code>	Add a task to an existing owned plan.	<code>plan_id: int</code> , <code>topic_id: int</code> , <code>description?: str</code>	<code>{task_id, plan_id, topic, description}</code>	INSERT <code>PracticeTask</code> .
<code>recommend_quiz</code>	Suggest a random active quiz for a topic.	<code>topic_id: int</code>	<code>{quiz_id, title, topic}</code> or <code>{quiz: null, message}</code>	Read-only.
<code>delete_practice_task</code>	Destructive. Remove a practice task.	<code>task_id: int</code> , <code>confirm: bool</code>	<code>{deleted_task_id}</code>	DELETE <code>PracticeTask</code> .
<code>mark_task_done</code>	Toggle a task's completion.	<code>task_id: int</code> , <code>is_done?: bool</code>	<code>{task_id, is_done}</code>	UPDATE <code>PracticeTask</code> .
<code>explain_topic</code>	AI tutor explanation of a topic, grounded in relevant course materials (or a synthesized primer).	<code>topic_id: int</code>	<code>{topic, explanation}</code>	Read-only.
<code>generate_practice_quiz</code>	Ephemeral, answer-revealed self-check quiz for a task or topic.	<code>task_id?: int</code> , <code>topic_id?: int</code> , <code>num_mcq?: int</code>	<code>{topic, task_id, practice_questions: [{question, choices, correct_answer}]}</code>	Read-only (no <code>Quiz</code> row created).
<code>summarize_material</code>	Personalized summary of one material in a chosen style.	<code>material_id: int</code> , <code>style?: enum</code>	<code>{material_id, material, style, summary}</code>	Read-only (never overwrites <code>Material.ai_summary</code>).
<code>ask_about_material</code>	AI tutor scoped to one material; 6 modes (<code>ask</code> / <code>explain_simply</code> / <code>give_example</code> / <code>explain_paragraph</code> / <code>hint</code> / <code>quiz_me</code>).	<code>material_id: int</code> , <code>mode?: enum</code> , <code>question?: str</code>	<code>{material_id, material, mode, tutor_reply}</code> or (mode= <code>quiz_me</code>) <code>{material, topic, practice_questions}</code>	Read-only.
<code>mark_material_done</code>	Toggle a material's studied status.	<code>material_id: int</code> , <code>is_done?: bool</code>	<code>{material_id, material, is_done}</code>	UPSERT <code>MaterialProgress</code> .

Tool Name	Description	Input Arguments	Output	DB Side-Effects
<code>get_learning_dashboard</code>	Learning Analytics: progress, quiz average, strongest/weakest topic, materials completed, study streak, AI narrative.	<i>(none)</i>	<code>{overall_progress_percent, quiz_average, strongest_topic, weakest_topic, completed_materials, total_materials, study_streak_days, insight}</code>	Read-only.
<code>get_recommendations</code>	Ranked, explainable "recommended for you" list.	<i>(none)</i>	<code>{recommendations: [...]}</code>	Read-only.
<code>start_adaptive_practice</code>	Begin an adaptive session on a topic at a mastery-derived difficulty.	<code>topic_id: int</code>	<code>{session_id, topic, difficulty, question, choices, question_number}</code>	UPDATE (deactivate prior sessions); INSERT <code>AdaptivePracticeSession</code> .
<code>submit_adaptive_answer</code>	Grade the current question, adjust difficulty, generate the next.	<code>selected_index: int</code>	<code>{feedback, was_correct, correct_answer, difficulty, next_question, next_choices, question_number, score}</code> (or <code>ended: true</code> if generation fails)	UPDATE <code>AdaptivePracticeSession</code> .
<code>end_adaptive_practice</code>	End the active session with a summary.	<i>(none)</i>	<code>{topic, questions_asked, questions_correct, final_difficulty}</code>	UPDATE <code>AdaptivePracticeSession</code> (<code>is_active=false</code>).

Instructor Agent Tools (17)

Tool Name	Description	Input Arguments	Output	DB Side-Effects
<code>get_course_analytics</code>	Aggregate analytics for an owned course: avg score, score histogram, topic breakdown, recent attempts, pending-essay count.	<code>course_id: int</code>	<code>{course, student_count, quiz_count, attempt_count, avg_score, topic_breakdown, score_distribution, recent_attempts, pending_essay_count, ai_summary, ai_summary_generated_at}</code>	Read-only.
<code>get_student_performance</code>	One student's attempt history and per-topic mastery within an owned course.	<code>course_id: int</code> , <code>student_id: int</code>	<code>{student, student_id, attempt_history, topics}</code>	Read-only.
<code>create_quiz</code>	Manually create an empty quiz for a topic.	<code>course_id: int</code> , <code>topic_id: int</code> , <code>title?: str</code>	<code>{quiz_id, title, topic}</code>	INSERT <code>Quiz</code> .

Tool Name	Description	Input Arguments	Output	DB Side-Effects
<code>generate_course_summary</code>	AI-narrated course overview cached on <code>Course.ai_summary</code> .	<code>course_id: int</code>	<code>{course, summary}</code>	UPDATE <code>Course</code> .
<code>cancel_quiz</code>	Destructive. Deactivate a quiz.	<code>quiz_id: int</code> , <code>confirm: bool</code>	<code>{quiz_id, title, is_active}</code>	UPDATE <code>Quiz</code> (<code>is_active=false</code> , <code>cancelled_at</code>).
<code>review_study_plan</code>	Approve/reject a student's pending plan.	<code>plan_id: int</code> , <code>decision: enum</code> , <code>note?: str</code>	<code>{plan_id, status}</code>	UPDATE <code>StudyPlan</code> ; notify student.
<code>upload_material</code>	Save an uploaded file as a <code>Material</code> and run text extraction.	<code>course_id: int</code> , <code>django_file</code> , <code>module_id?: int</code> , <code>lesson_id?: int</code> , <code>title?: str</code>	<code>{material_id, title, kind, extraction_status, chars_extracted}</code>	INSERT <code>Material</code> ; UPDATE (<code>extracted_text</code> , <code>extraction_status</code>).
<code>generate_summary_from_material</code>	AI summary of one material, cached on <code>Material.ai_summary</code> .	<code>material_id: int</code>	<code>{material_id, summary}</code>	UPDATE <code>Material</code> .
<code>generate_quiz_from_material</code>	AI quiz generated from one material's extracted text.	<code>material_id: int</code> , <code>topic_id: int</code> , <code>num_mcq?: int</code> , <code>num_essay?: int</code> , <code>title?: str</code>	<code>{quiz_id, title, mcq_count, essay_count}</code>	INSERT <code>Quiz</code> , <code>Question</code> (×N), <code>Choice</code> (per MCQ).
<code>generate_quiz_for_topic</code>	AI quiz for a topic using all relevant course materials as context (or a synthesized primer).	<code>course_id: int</code> , <code>topic_id: int</code> , <code>num_mcq?: int</code> , <code>num_essay?: int</code> , <code>title?: str</code>	<code>{quiz_id, title, mcq_count, essay_count}</code>	INSERT <code>Quiz</code> , <code>Question</code> (×N), <code>Choice</code> (per MCQ).
<code>add_question_to_quiz</code>	Manually author one question (MCQ needs ≥2 choices, exactly 1 correct).	<code>quiz_id: int</code> , <code>text: str</code> , <code>question_type?: enum</code> , <code>choices?: list</code>	<code>{question_id, quiz_id, question_type}</code>	INSERT <code>Question</code> ; INSERT <code>Choice</code> (×N, if MCQ).
<code>grade_essay_response</code>	Manually grade one essay/short-answer response (0–1).	<code>response_id: int</code> , <code>score: float</code> , <code>feedback?: str</code>	<code>{response_id, attempt_id, attempt_status, attempt_score}</code>	UPDATE <code>QuestionResponse</code> , <code>QuizAttempt</code> ; notify student.
<code>delete_material</code>	Destructive. Permanently delete a material and its file.	<code>material_id: int</code> , <code>confirm: bool</code>	<code>{deleted_material_id, title, kind, note}</code>	DELETE <code>Material</code> (+ file); generated quizzes retained.
<code>get_at_risk_students</code>	Students below a mastery threshold, with weakest topic.	<code>course_id: int</code> , <code>threshold?: float</code>	<code>{course, threshold, at_risk_students: [...]}</code>	Read-only.
<code>send_announcement</code>	Broadcast a notification to every enrolled student.	<code>course_id: int</code> , <code>message: str</code>	<code>{course, recipient_count, message}</code>	INSERT <code>Notification</code> (×N).

Tool Name	Description	Input Arguments	Output	DB Side-Effects
<code>analyze_class_performance</code>	Hardest topic, class average, at-risk count, plus an AI-written recommended action.	<code>course_id: int</code>	<code>{most_difficult_topic, most_difficult_topic_avg_mastery, average_score, at_risk_count, student_count, insight, ...}</code>	Read-only.
<code>generate_remedial_quiz</code>	AI quiz auto-targeted at the course's current hardest topic (or an explicit topic).	<code>course_id: int</code> , <code>topic_id?: int</code> , <code>num_mcq?: int</code> , <code>num_essay?: int</code> , <code>title?: str</code>	<code>{quiz_id, title, topic, mcq_count, essay_count}</code>	<code>INSERT Quiz</code> , <code>Question</code> (×N), <code>Choice</code> (per MCQ).

8.3 Security Boundaries

Dynamic permission checks are performed **before** any tool executes its business logic, in a fixed order for every call:

1. **Role gate.** `if not user.is_student: raise ToolPermissionError(...)` (Student Agent tools) or the `is_instructor` equivalent (Instructor Agent tools) — the very first line of every tool function body.
2. **Ownership/enrollment gate.** Every subsequent database lookup is filtered by the calling user's relationship to the target row (`instructor=user`, `students=user`, `course__instructor=user`, `plan__student=user`, `attempt__quiz__course__instructor=user`, etc.), returning a `ToolValidationError` ("not found or you do not own it") rather than a distinguishable "exists but forbidden" response — this deliberately prevents the tool from ever confirming the *existence* of a resource the caller cannot access, closing an information-disclosure side channel that a naive "403 vs. 404" split would open.
3. **Destructive-action gate.** Applied by the `@agent_tool(destructive=True)` decorator wrapper, before the tool function's own body runs at all: the first call without `confirm=true` never reaches the business logic and only records a `pending_confirm` audit row.
4. **Value validation.** Only after the above three gates pass does the tool validate its own argument values (e.g., `grade_essay_response` 's score must parse as a float in `[0, 1]`; `add_question_to_quiz` 's MCQ must have ≥ 2 choices with exactly one correct).

This ordering guarantees that a malformed or adversarial argument set can never be used to probe for the existence of unauthorized data, and that a destructive action can never execute on the strength of a single LLM-inferred call, however confidently worded.

8.4 Sample Agent Traces

Trace 1 — Student Agent, LLM (Gemini) tool-calling path, English:

```
{
  "conversation_id": "sa-2f9c1e",
  "agent": "student_agent",
  "user": {"id": 41, "username": "sara_m", "role": "student"},
  "provider": "gemini-3.8-flash",
```

```

"turns": [
  {
    "role": "user",
    "parts": [{"text": "quiz me on database normalization, easy stuff to start"}]
  },
  {
    "role": "model",
    "parts": [
      {
        "function_call": {
          "name": "generate_practice_quiz",
          "args": {"topic_id": 14, "num_mcq": 4}
        }
      }
    ]
  },
  {
    "role": "user",
    "parts": [
      {
        "function_response": {
          "name": "generate_practice_quiz",
          "response": {
            "ok": true,
            "data": {
              "topic": "Database Normalization",
              "task_id": null,
              "practice_questions": [
                {
                  "question": "What is the primary goal of 1NF?",
                  "choices": ["Eliminate transitive dependencies", "Ensure atomic column
values", "Reduce foreign keys", "Increase index count"],
                  "correct_answer": "Ensure atomic column values"
                },
                {
                  "question": "A table in 2NF but not 3NF exhibits which issue?",
                  "choices": ["Partial dependency", "Transitive dependency", "Multivalued
dependency", "No primary key"],
                  "correct_answer": "Transitive dependency"
                }
              ]
            }
          }
        }
      }
    ]
  },
  {
    "role": "model",
    "parts": [{"text": "Here are 2 quick easy questions on Database Normalization:\n\n1. What
is the primary goal of 1NF?\n  a) Eliminate transitive dependencies  b) Ensure atomic column
values  c) Reduce foreign keys  d) Increase index count\n\n2. A table in 2NF but not 3NF
exhibits which issue?\n  a) Partial dependency  b) Transitive dependency  c) Multivalued
dependency  d) No primary key\n\nWant harder ones next, or should I start an adaptive session
so difficulty adjusts automatically?"}]
  }
],
"agent_action_log_rows": [
  {
    "id": 5821,
    "user_id": 41,

```

```

    "agent": "student_agent",
    "tool_name": "generate_practice_quiz",
    "params": {"topic_id": 14, "num_mcq": 4},
    "result": {"ok": true, "data": {"topic": "Database Normalization", "task_id": null,
"practice_questions": "[...2 items...]"}},
    "status": "success",
    "timestamp": "2026-09-14T10:22:07Z"
  }
}
}

```

Trace 2 — Instructor Agent, offline NLU fallback path (Gemini rate-limited), Egyptian Arabic, destructive action requiring confirmation:

```

{
  "conversation_id": "ia-77ab30",
  "agent": "instructor_agent",
  "user": {"id": 7, "username": "dr_hossam", "role": "instructor"},
  "provider": "offline_nlu (Gemini 429 on all models in chain)",
  "turns": [
    {
      "role": "user",
      "text": "امسح الملف بتاع lecture3.pdf"
    },
    {
      "role": "agent_internal",
      "note": "llm_available() -> True; run_tool_loop raised LLMUnavailableError('all models cooling down'); falling back to agents.nlu.route_instructor"
    },
    {
      "role": "nlu_router",
      "resolved_intent": "delete_material",
      "resolved_entities": {"material_id": 302, "material_title": "lecture3.pdf"},
      "note": "Egyptian Arabic keyword match on 'امسح' (delete) + fuzzy filename match; only one material named lecture3.pdf in this instructor's courses"
    },
    {
      "role": "tool_call",
      "name": "delete_material",
      "args": {"material_id": 302}
    },
    {
      "role": "tool_result",
      "output": {
        "ok": false,
        "requires_confirmation": true,
        "message": "'delete_material' is a destructive action. Call again with confirm=true to proceed."
      }
    },
    {
      "role": "agent",
      "text": "⚠ The AI connection isn't configured / is rate-limited right now, but I still found and matched your request.\n\n'delete_material' is a destructive action. Call again with confirm=true to proceed."
    },
    {
      "role": "user",
      "text": "تمام، اتأكد وامسحه"
    }
  ]
}

```

```

    },
    {
      "role": "tool_call",
      "name": "delete_material",
      "args": {"material_id": 302, "confirm": true}
    },
    {
      "role": "tool_result",
      "output": {
        "ok": true,
        "data": {
          "deleted_material_id": 302,
          "title": "lecture3.pdf",
          "kind": "pdf",
          "note": "2 quiz(zes) generated from this material were kept (they are independent
records)."
        }
      }
    },
    {
      "role": "agent",
      "text": "Deleted 'lecture3.pdf'. Note: 2 quiz(zes) generated from this material were kept
(they are independent records).\"
    }
  ],
  "agent_action_log_rows": [
    {
      "id": 6104,
      "user_id": 7,
      "agent": "instructor_agent",
      "tool_name": "delete_material",
      "params": {"material_id": 302},
      "result": {"prompt": "'delete_material' is a destructive action. Call again with
confirm=true to proceed."},
      "status": "pending_confirm",
      "timestamp": "2026-09-14T11:03:41Z"
    },
    {
      "id": 6105,
      "user_id": 7,
      "agent": "instructor_agent",
      "tool_name": "delete_material",
      "params": {"material_id": 302, "confirm": true},
      "result": {"deleted_material_id": 302, "title": "lecture3.pdf", "kind": "pdf", "note": "2
quiz(zes) generated from this material were kept (they are independent records).\"},
      "status": "success",
      "timestamp": "2026-09-14T11:03:58Z"
    }
  ]
}

```

9. GitHub & Development Strategy

9.1 Branching Model (GitFlow)

Branch	Purpose	Merge Policy
<code>main</code>	Always production-deployable. Tagged at every release (<code>vX.Y.Z</code>).	Only merged into via <code>hotfix/*</code> or a <code>release/*</code> branch; never committed to directly.
<code>develop</code>	Integration branch for the next release; always contains the latest completed features.	<code>feature/*</code> branches merge here via Pull Request, squash-merged after review.
<code>feature/*</code>	One feature or tool per branch, e.g. <code>feature/adaptive-practice-difficulty</code> , <code>feature/generate-remedial-quiz</code> , <code>feature/offline-nlu-arabic</code> .	Branched from <code>develop</code> ; merged back into <code>develop</code> ; deleted after merge.
<code>hotfix/*</code>	Urgent production fixes, e.g. <code>hotfix/mcq-regex-unicode</code> , <code>hotfix/gemini-429-retry</code> .	Branched from <code>main</code> ; merged into both <code>main</code> and <code>develop</code> to keep them in sync.
<code>release/*</code>	Stabilization window before a <code>main</code> merge (freeze, final QA, changelog).	Branched from <code>develop</code> ; merged into <code>main</code> and back into <code>develop</code> .

9.2 Commit Rules (Conventional Commits)

All commits shall follow the Conventional Commits specification: `<type>(<scope>): <short summary>` , with `<scope>` naming the affected Django app or module where applicable.

Type	Use For	Example
<code>feat</code>	A new user-facing capability or tool.	<code>feat(agents): add generate_remedial_quiz instructor tool</code>
<code>fix</code>	A bug fix, including resilience/regression fixes.	<code>fix(courses): unicode-aware word regex for Arabic MCQ generation</code>
<code>docs</code>	Documentation-only changes.	<code>docs(readme): document AI_OFFLINE_MODE flag</code>
<code>refactor</code>	Code change with no behavior change.	<code>refactor(llm_client): extract model chain into its own function</code>
<code>test</code>	Adding or updating tests only.	<code>test(agents): add 429/404/403 circuit breaker coverage</code>
<code>chore</code>	Tooling, dependency bumps, CI config.	<code>chore(deps): bump google-genai to 1.x</code>
<code>perf</code>	Performance improvement with no functional change.	<code>perf(courses): aggregate topic mastery with values().annotate()</code>

Breaking changes are flagged with a `!` after the type/scope (e.g., `feat(agents)!: rename course_id param to courseId`) and a `BREAKING CHANGE:` footer describing the migration path.

9.3 Issue Board Organization

Kanban columns: `Backlog` → `Ready` → `In Progress` → `In Review` → `QA / Verify` → `Done`.

Label taxonomy:

- **Type:** `type:feature`, `type:bug`, `type:chore`, `type:docs`, `type:security`
- **Module:** `module:accounts`, `module:courses`, `module:planning`, `module:agents`, `module:chatbot`, `module:notifications`
- **Priority:** `priority:critical` (production-down, e.g. an unhandled 500 from an LLM provider failure), `priority:high`, `priority:medium`, `priority:low`
- **AI-specific:** `ai:llm-orchestration`, `ai:offline-nlu`, `ai:prompt-tuning`, `ai:resilience` (circuit breaker / fallback chain work)
- **RBAC/security:** `security:rbac`, `security:audit-log`, `security:prompt-injection`

Task prioritization lifecycle: every issue enters `Backlog` with a `type:*` and `module:*` label; grooming assigns a `priority:*` label and moves it to `Ready`; a contributor self-assigns and moves it to `In Progress` on branching; opening a Pull Request against `develop` moves it to `In Review`; passing the module's test suite (e.g., `python manage.py test agents` for anything touching `agents/`) and a peer approval moves it to `QA / Verify`; manual verification against the acceptance criteria in the issue description (including, for AI-touching issues, a verification pass with `AI_OFFLINE_MODE=1` to confirm the offline fallback path was not broken) moves it to `Done`, at which point the branch is deleted.

9.4 Reference Development Workflow

The following commands (drawn from the project's own `README.md`) constitute the canonical local setup and verification loop, to be reflected in the CI pipeline running against every Pull Request into `develop`:

```
pip install -r requirements.txt
cp .env.example .env                # set GEMINI_API_KEY if available
docker compose up -d                # PostgreSQL 16 on localhost:5432
python manage.py migrate
python manage.py seed_demo           # demo courses/users/materials
python manage.py ai_status --test    # verify LLM connectivity & model chain
python manage.py test agents         # circuit breaker + offline-NLU regression suite
python manage.py runserver
```

CI shall additionally run with `AI_OFFLINE_MODE=1` set at least once per pipeline, so that a Pull Request cannot merge a change that silently depends on live LLM connectivity for its test coverage — the platform's core reliability guarantee (NFR-3.4.2, NFR-3.4.5) is exercised on every build, not only in production incidents.

End of Software Requirements Specification.